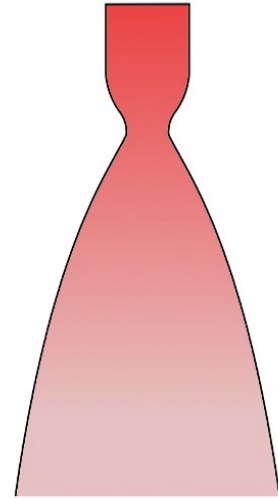


Rocket Propulsion Analysis



Version 4.0.0

User Manual

Cologne, Germany – 2024
www.rocket-propulsion.com

Table of Contents

Introduction.....	6
Key Features.....	6
Calculation Methodology.....	6
Technical Specifications.....	6
RPA editions.....	7
System requirements.....	7
Microsoft Windows.....	7
Apple Mac.....	7
Linux.....	7
Installation on Microsoft Windows.....	7
Installation on Linux.....	7
Running RPA.....	8
Using Graphical User Interface.....	8
Using Scripting utility.....	9
Configuration Files.....	10
Create a Configuration File.....	10
Load a Configuration File.....	10
Save the Current Analysis.....	10
Save As Another File.....	10
Recent Files.....	10
Example Configuration Files.....	11
Migrating Configuration Files.....	11
Engine Definition.....	11
Nozzle Flow Model.....	14
Nozzle conditions.....	14
Delivered performance.....	16
Ambient conditions.....	17
Throttle settings.....	17
Thrust Chamber Size.....	18
Thrust Chamber Sizing.....	18
Combustion Chamber Geometry.....	19
Nozzle Geometry.....	19
Thermal Analysis.....	20
Cooling segments.....	21
Radiation Cooling.....	22
Regenerative Cooling.....	22
Coolant definition.....	23
Coaxial Shell Jacket Design.....	24
Tubular Wall Jacket Design.....	25
Channel Wall Jacket Design.....	26
Film Cooling.....	28
Thrust chamber with multiple cooling segments.....	28
Thermal Barrier Coating Layer.....	29
Starting Analysis.....	29

Rocket Propulsion Analysis v.4.0.0

Results.....	30
Thermodynamics.....	30
Combustion products.....	32
Performance.....	33
Altitude performance.....	35
Throttled performance.....	37
Nested Analysis.....	38
Propellant Analysis.....	40
Thrust chamber size.....	42
Thermal Analysis.....	44
Propellant Feed System (Cycle Analysis).....	45
Thermodynamic Database Editor.....	46
Preferences.....	49
Input and Output Units.....	50
Scripting API.....	52
API Reference.....	52
Generic built-In Functions / Objects.....	52
Object File.....	54
Object ConfigFile.....	55
Object Units.....	57
Object Database.....	60
Object Species.....	60
Object Mixture.....	62
Object Propellant.....	64
Object Equilibrium.....	65
Object Derivatives.....	69
Object Product.....	71
Object Combustor.....	73
Object Chamber.....	75
Object ChamberStationConditions.....	78
Object TheoreticalPerformance.....	82
Object TheoreticalThrottlingPerformance.....	84
Object ChamberDesign.....	86
Object NozzleDesign.....	87
Object DXF.....	88
Object HeatTransferAnalysis.....	88
Object PropulsionAnalysis.....	88
Object AltitudePerformance.....	89
Object PropellantConfiguration.....	90
Object ChamberConfiguration.....	91
Object ChamberDesignConfiguration.....	93
Object NozzleDesignConfiguration.....	94
Object ChamberDesignResults.....	95
Object NozzleDesignResults.....	96
Object HeatTransferConfiguration.....	97
Object FilmCoolingSlot.....	98

Rocket Propulsion Analysis v.4.0.0

<i>Object WallCoolingSegment</i>	98
Object HeatTransferAnalysisResults.....	102
Object Units.....	102
Object HeatTransferStation.....	102
Object WallCooling.....	103
Attachments.....	104
Equivalence Ratio calculation.....	104
Oxygen Balance calculation.....	105

Introduction

Rocket Propulsion Analysis (RPA) is a versatile and user-friendly multi-platform tool designed for predicting the performance of rocket engines. It features an intuitive graphical user interface (GUI) that conveniently groups input parameters and analysis results, making it accessible for both novice and experienced users.

Key Features

- **Intuitive GUI:** The user interface is designed for ease of use, facilitating straightforward input and clear display of results.
- **Comprehensive Chemical Species Library:** RPA utilizes an expandable chemical species library based on the NASA Glenn thermodynamic database. This includes data for a wide range of fuels and oxidizers such as liquid hydrogen, liquid oxygen, kerosene, hydrogen peroxide, and MMH.
- **Customizable Propellant Components:** Users can define new propellant components or import components from PROPEP or CEA2 species databases using the embedded species editor.
- **Performance Prediction:** By providing essential engine parameters such as combustion chamber pressure, propellant components, and nozzle parameters, RPA calculates the chemical equilibrium composition of combustion products, determines their thermodynamic properties, and predicts theoretical rocket performance.
- **Design Applications:** The results can be utilized in designing combustion chambers, gas generators, and preburners for liquid propellant rocket engines.

Calculation Methodology

RPA employs a robust and industry-accepted Gibbs free energy minimization approach to:

- Determine combustion composition
- Analyze nozzle flows with shifting and frozen chemical equilibrium
- Calculate engine performance for both finite- and infinite-area combustion chambers

Technical Specifications

RPA is written in C++ programming language using libraries: Qt 6, Qwt, Google protobuf, cryptopp.

The program is developed and distributed by RP Software+Engineering UG. The development team can be contacted by email: contact@propulsion-analysis.com

RPA editions

Starting with version 4, RPA is distributed in only one edition (formerly known as Standard Edition).

System requirements

Microsoft Windows

Operating Systems:

- Windows 11, Windows 10, Windows XP (64-bit Editions).

Any computer that runs with the mentioned operating systems.

Apple Mac

- Not supported

Linux

RPA will not run without the following libraries:

- Qt 6.0 or higher

Installation on Microsoft Windows

RPA for Microsoft Windows is distributed as a compressed bundle (ZIP) for x86-64 architecture.

RPA for Windows depends on Qt 6 and MS VC++ 2019 run-time libraries. If computer does not have it installed, the bundle that includes all required components should be used for installation.

Installation steps:

- Extract files from the compressed bundle (ZIP) into selected directory
- Start the program by executing the command `RPA.exe`

The user does not need administrator privileges for installing the software.

To uninstall the application, remove the RPA installation directory.

Installation on Linux

RPA for Linux is distributed as a compressed bundle (TAR.GZ) for x86-64 architecture.

RPA for Linux depends on Qt 6 run-time libraries.

To install the program

- Extract files from the compressed bundle (TAR.GZ) into selected directory
- Start the program by executing the shell script `RPA.exe`

The user does not need administrator privileges for installing the software to the HOME directory.

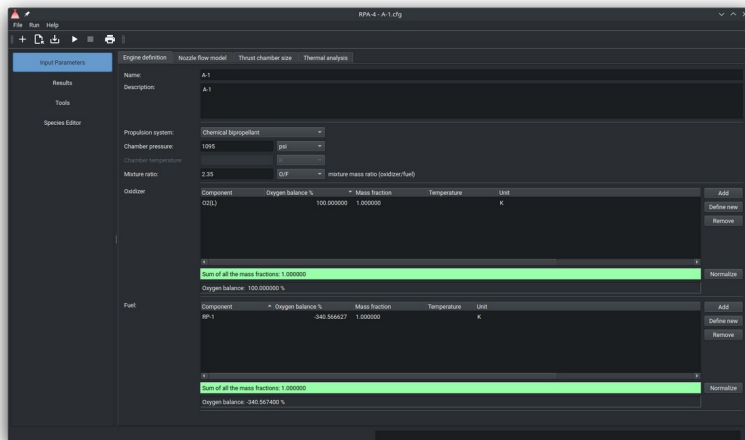
To uninstall the application, remove the RPA installation directory.

Running RPA

Using Graphical User Interface

After launching RPA by starting the program `RPA.exe`, the main window will appear. The main window includes a menu bar, toolbar, and working area designed for ease of use and efficient navigation.

Working area consists of several screens which conveniently group the input parameters and analysis results. The desired screen can be activated by clicking the corresponding button on the list located on the left side of the main window. The user can resize the list and working area by dragging the vertical bar between them to the right or left. This allows the user to enlarge the list and narrow the working area, or vice versa, according to preference.



Main window

Using Scripting utility

The scripting utility of RPA allows users to execute custom problems and initiate analyses from the command line.

It supports both interactive and batch modes, providing flexibility for different use cases.

- **Interactive Mode:** Allows users to interact with the program, input commands, and receive immediate feedback.
- **Batch Mode:** Enables users to execute a series of commands automatically, ideal for running pre-defined scripts or multiple analyses without manual intervention.

To start the scripting utility, open a command line interface, navigate to RPA installation directory, and type the following command:

```
RPA-Script.exe
```

When executed without these arguments `--input` or `--stream`, the scripting utility is started in the interactive mode, displaying the prompt `rpa>`, inviting the user to type valid commands.

Type the command "exit" to stop the utility running in the interactive mode. See *Scripting Built-In Commands* and *Scripting API Reference* to get more information about available commands.

To start the scripting utility in the batch mode, use the command-line arguments `--input` or `--stream` to specify the code to be executed in the batch mode. After completion, the scripting utility prints out the results in the console window, writes the log file (if applicable), and exits.

See *Scripting API Reference* to get more information JavaScript API used to create the batch scripts.

The available command-line arguments are shown below:

Option	Value	Description
<code>-h</code> or <code>--help</code>		Display help
<code>-v</code> or <code>--version</code>		Display version information
<code>--std_thermo</code>	FILE	Path to standard thermodynamic database file. Default is resources/thermo.inp
<code>-t</code> or <code>--usr_thermo</code>	FILE	Path to user-defined thermodynamics database file. Default is resources/usr_thermo.inp
<code>--std_properties</code>	FILE	Path to properties database file. Default is resources/properties.inp
<code>-p</code> or <code>--usr_properties</code>	FILE	Path to user-defined properties database file. Default is resources/properties.inp
<code>--std_transport</code>	FILE	Path to transport properties database file. Default is resources/trans.inp
<code>--input</code>	FILE	Path to script file to be executed

Option	Value	Description
--debug		Print out debug log to console. Can be useful for troubleshooting
--stream		Execute script code from input stream

Configuration Files

The input data for analysis problems is stored in the configuration files with extension `.cfg`. The configuration file has a JSON format, that can be viewed/edited in any ASCII text editor.

Create a Configuration File

- Click **File** and then **New** in the menu bar, or
- Click the **New** icon on the toolbar.

Load a Configuration File

- Click **File** and then **Open** in the menu bar, or
- Click the **Open** icon on the toolbar.
- Select the file you wish to open.

If another configuration file is already open, or a new one has been created, a dialog window will appear to confirm the opening of another configuration file.

Save the Current Analysis

- Click **File** and then **Save** in the menu bar.
- If the current analysis problem is new, you will be prompted to specify a file name or select an existing file to overwrite. Otherwise, the data will be saved to the previously opened file.

Save As Another File

- Click **File** and then **Save As...** in the menu bar.

Recent Files

- The last 15 used files are listed at the bottom of the **File** menu in the menu bar for quick access.
- The most recently used configuration file is automatically opened when the program starts.

Example Configuration Files

- The program includes several example configuration files located in the directory *examples* inside the installation directory.

Migrating Configuration Files

Configuration files created for earlier versions of RPA are not compatible with the JSON format used in RPA 4. To convert these configuration files, use the provided script `examples/scripting/migrate_cfg.js` with two arguments:

Input file path: Path to the old configuration file.

Output file path: Path where the new JSON configuration file will be saved.

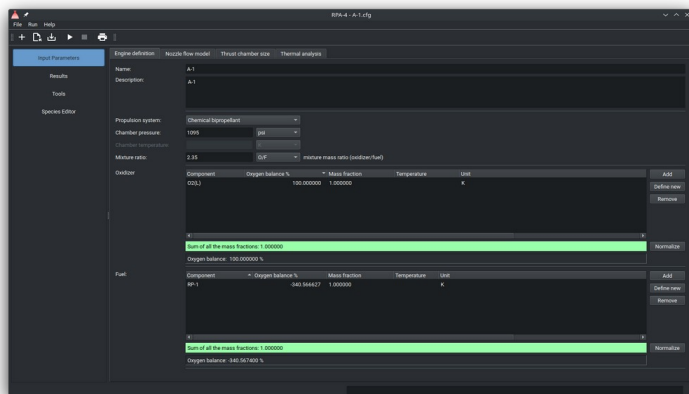
Example usage:

```
RPA-Script.exe --input ./examples/scripting/migrate_cfg.js
old_config.cfg new_config.cfg
```

Engine Definition

The screen *Engine Definition* is used to define the parameters:

- Case name and description
- Propellant specification
 - Propulsion system:
 - Chemical bipropellant propulsion,
 - Chemical monopropellant propulsion, or
 - Thermal propulsion
 - Propellant composition



Engine Definition screen

For bipropellant systems, the lists for both *Oxidizer* and *Fuel* are enabled, as well as the fields for specifying a mixture ratio. The mixture ratio can be specified in one of two ways:

O/F ratio: The ratio of oxidizer flow rate to fuel flow rate.

Oxidizer excess coefficient: The ratio of the desired O/F to the stoichiometric O/F.

The "optimum" type of mixture ratio can be used to optimize the mixture ratio for maximum specific impulse. The obtained optimum O/F ratio will be displayed on the Results/Thermodynamics screen.

For monopropellant systems, only the list of *Propellant* components is enabled. The *Fuel* list and the fields for specifying the mixture ratio are disabled.

Note: For the thermodynamic analysis, the only difference between a bipropellant system and a monopropellant system (that contains more than one species) is how the mixture composition is defined. Any bipropellant system (as well as three- or more propellant systems) can be defined as a monopropellant system by specifying the proper mass fraction for each component on the list.

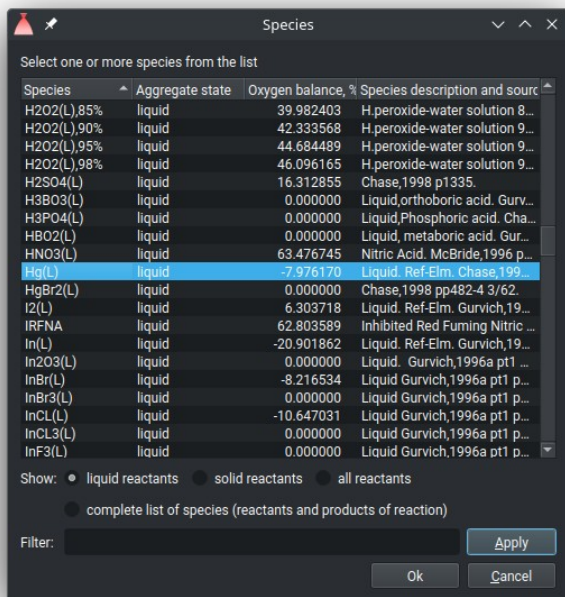
For example, the following "monopropellant" configuration is equivalent to the bipropellant one with $O/F=0.7245/0.2755=2.63$:

Propulsion system:	Chemical monopropellant			
Chamber pressure:	1095	psi		
Chamber temperature:		K		
Mixture ratio:	2.35	O/F	mixture mass ratio (oxidizer/fuel)	

Propellant	Component	Oxygen balance %	Mass fraction	Temperature	Unit	
	RP-1	-340.566627	0.724500		K	Add
	O2(L)	100.000000	0.275500		K	Define new
						Remove
Sum of all the mass fractions: 1.000000						Normalize
Oxygen balance: -219.190882 %						

Each component list (Oxidizer and Fuel, or Propellant) may contain one or more species, displayed on the single row. To add species to the list, click the button **Add** on the right side of the corresponding list.

After clicking the button **Add**, the dialog window "Components" appears.



The user can filter the list in the dialog window using a filter pattern. The filter pattern is applied to columns "Species" and "Species description and source data references".

Mark the check box "complete list of species (reactants and products of reaction)" to see all species, including atomized and/or ionized products of the reaction. Leave it unmarked to see only possible propellant components.

- **To add species:**
Select one or more species from the list.
Click the button **OK**.
- **To cancel:**
Click the button **Cancel** to leave without adding any species.
- **To remove species:**
Click the button **Remove** to delete the selected species from the list.
- **To add new species:**
Click the button **Define new**.

Each component on the list features four parameters:

- Component name
- Oxygen balance
- Mass fraction
- Temperature of the component

Component	Oxygen balance %	Mass fraction	Temperature	Unit
RP-1	-340.566627	0.724500		K
O2(L)	100.000000	0.275500		K

The parameter *Mass fraction* is required. To modify the value, double click the corresponding cell of the row.

When composing a multi-component mixture, ensure that the sum of all the mass fractions of components on the same list equals 1. Each list includes an automatic mass fraction checker, which displays the current sum at the list footer. If the sum is correct, the footer's background color is light-green; if incorrect, the color is light-red:

Sum of all the mass fractions: 0.979500

The initial temperature of the component can be specified in the column “*Temperature*”. This is an optional parameter; if not specified, the default standard temperature will be assigned automatically. To add/modify the value or unit, double click the corresponding cell of the row.

Nozzle Flow Model

The code can either calculate the combustion parameters in the combustion chamber, or perform the complete engine performance analysis, analyzing the nozzle flow, performing the thrust chamber sizing and thermal analysis.

Clear the check box “*Nozzle conditions*”, in order to perform combustion analysis only, or mark it to perform the nozzle flow analysis followed by thrust chamber sizing and thermal analysis (if configured):

Engine definition

Nozzle flow model

Thrust chamber

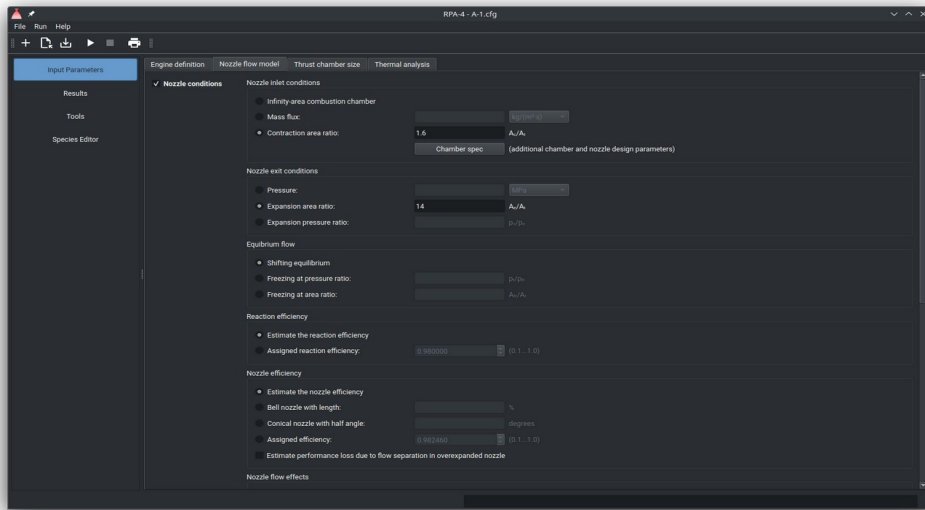
☒ Nozzle conditions

Nozzle inlet conditions

Nozzle conditions

To solve the nozzle flow problem, define at least one of the nozzle exit conditions by specifying one of the following three parameters:

- Nozzle exit pressure
- Nozzle expansion area ratio
- Nozzle expansion pressure ratio



Nozzle conditions

The code can calculate the performance for a combustion chambers with fixed or infinite cross section area. The default model assumes the infinite cross section area of the chamber. To switch to the fixed-area model, mark the check box "*Nozzle inlet conditions*", and then specify the chamber contraction area ratio (A_c/A_t) or mass flux ($\dot{m}/A_c$) in the combustion chamber.

If the nozzle inlet and exit conditions are specified, they are also used for thrust chamber sizing, along with additional parameters provided on the screen **Thrust chamber size**.

The code can calculate the performance for the shifting and frozen chemical equilibrium in the nozzle. The default model assumes the shifting chemical equilibrium in the whole nozzle. To switch to the frozen chemical equilibrium model, mark one of the two check boxes

- *Freezing at pressure ratio*
- *Freezing at area ratio*

and then specify the value for selected condition. The frozen chemical equilibrium model will be applied to all nozzle stations downstream of the specified conditions: expansion pressure ratio or expansion area ratio.

Rocket Propulsion Analysis v.4.0.0

Equilibrium flow

- Shifting equilibrium
- Freezing at pressure ratio: p_0/p_∞
- Freezing at area ratio: A_{00}/A_∞

By default, the code calculates the parameters at following nozzle stations:

- nozzle inlet
- nozzle throat
- nozzle exit

Additional nozzle stations can be added by pressing the button **Add** on the right side of the list “Nozzle stations”, and defining the conditions by double-clicking the corresponding cells of the row:

Nozzle stations	Condition type	Location	Value	Unit	
	Area ratio	Nozzle inlet	2		
	Pressure ratio		1.2		
	Pressure ratio		2		
	Pressure		2	atm	
	Area ratio	Nozzle exit	10		

Delivered performance

The code calculates both theoretical and estimated delivered engine performance. The user can define *Reaction efficiency* and *Nozzle efficiency*.

Reaction efficiency

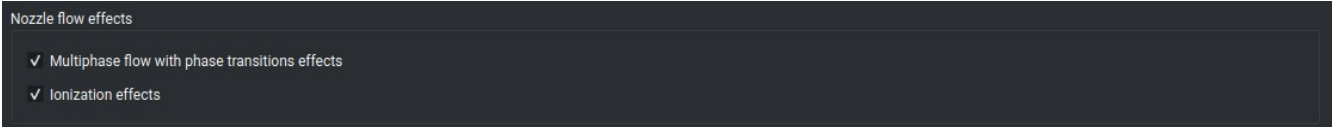
- Estimate the reaction efficiency
- Assigned reaction efficiency: 0.980000 (0.1...1.0)

Nozzle efficiency

- Estimate the nozzle efficiency
- Bell nozzle with length: %
- Conical nozzle with half angle: degrees
- Assigned efficiency: 0.982460 (0.1...1.0)
- ✓ Estimate performance loss due to flow separation in overexpanded nozzle

Reaction and nozzle efficiency

The code can assess effects of multiphase flow and ionization:



Switch off the flag **Multiphase flow and phase transition** in order to suppress the calculation of multiphase flow effects. Note that for the most of solid propellant problems this flag should be switched on.

Switch off the flag **ionization effects** in order to suppress the calculation of species ionization effects.

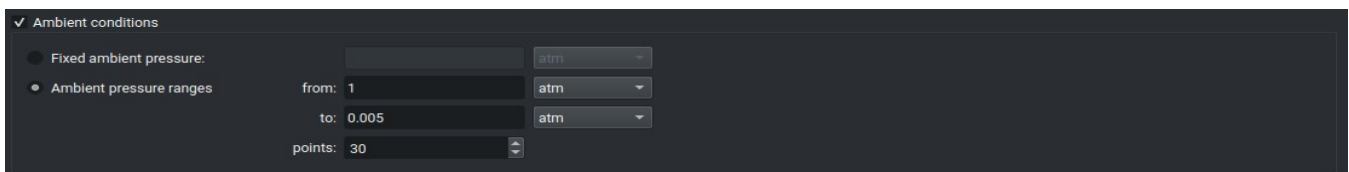
Ambient conditions

By default, the code calculates the performance of the thrust chamber at the following ambient conditions specified by ambient pressure (p_a):

- Sea level (SL) conditions ($p_a=1 \text{ atm}$, or 14.7 psi)
- Optimum nozzle expansion ($p_e=p_a$)
- Vacuum conditions ($p_a=0$)

To calculate the performance at the additional ambient conditions, the user can specify

- fixed ambient pressure
- range of ambient pressures given as high and low range values as well as number of steps between.



Ambient conditions

Throttle settings

By default, the code calculates the performance of the thrust chamber assuming the propellant flow rate does correspond to the nominal thrust.

To calculate the performance at the additional throttling values, the user can specify

- fixed throttle value
- range of throttle values given as high and low range values.

✓ Throttle settings

☒ Fixed throttle value: 1.0000 1/nom
☐ Throttle values range
 min: 0.7000 1/nom
 max: 1.2000 1/nom
 points: 10

Throttle settings

The throttling value is a ratio of the propellant flow rate at desired throttling setting to the flow rate at the nominal thrust. Throttling value for the nominal trust $R=1.0$

Thrust Chamber Size

Thrust Chamber Sizing

The code can estimate the size of the combustion chamber and nozzle matching one of the following design criteria:

- Nominal thrust at a certain ambient pressure
- Nominal mass flow rate
- Nozzle throat diameter

The input of parameters on the screen is only enabled if the flags "*Thrust chamber size*" is switched on:

Engine definition Nozzle flow model **Thrust chamber size** Thermal analysis

✓ **Thrust chamber size** Thrust chamber matches the specified requirements

☒ Engine nominal thrust: 100 kN
 at ambient pressure: 1 atm
☐ Engine propellant mass flow rate: 136 kg/s total at 100% throttle
☐ Chamber throat diameter: 136 mm
 Number of chambers: 1

Chamber and nozzle sizing parameters

If the specified number of chambers is greater than 1, the given thrust is a total engine thrust, and the given mass flow rate is a total mass flow rate. The thrust or mass flow rate of a single thrust chamber is obtained by dividing the design parameter value by the number of chambers.

If the ambient pressure is not specified, it is assumed that the engine nominally operates in vacuum condition ($p_a=0$).

Combustion Chamber Geometry

Set of parameters is used to specify the size of a cylindrical combustion chamber from the injector plate to the nozzle throat.

Chamber geometry

Cylindrical chamber

Chamber length (L_c or L^*):

0.31

m

☒ characteristic length L^*
☐ length L_c

Contraction area ratio:

Specify

Contraction angle b :

35

degrees

R_c/R_{max} :

1.0000

R_c/R_c :

0.8

Chamber geometry parameters

A simplified drawing with labeled dimensions on the right side helps to identify the meaning of parameters.

Nozzle Geometry

Set of parameters is used to specify the size of a cylindrical nozzle from the nozzle throat to the nozzle exit.

Nozzle geometry

☒ Conical nozzle with half angle:

degrees

R_n/R_c :

0.382

☒ Parabolic contour with fixed expansion area ratio A_e/A_t

☐ Initial parabola angle T_n :

degrees

☐ Final parabola angle T_e :

degrees

☐ Relative length $L_e/c15$:

80

%

☒ Truncated ideal contour (TIC)

☒ Maximum thrust at fixed expansion area ratio A_e/A_t
☐ Fixed nozzle expansion area ratio A_e/A_t and length L_e

Nozzle length L_e/D_t :

5.00

Surface temperature T_w/T_0 :

0.30

☒ Apply calculated nozzle efficiency to performance analysis

Chamber geometry parameters

Available options to design the nozzle shape include:

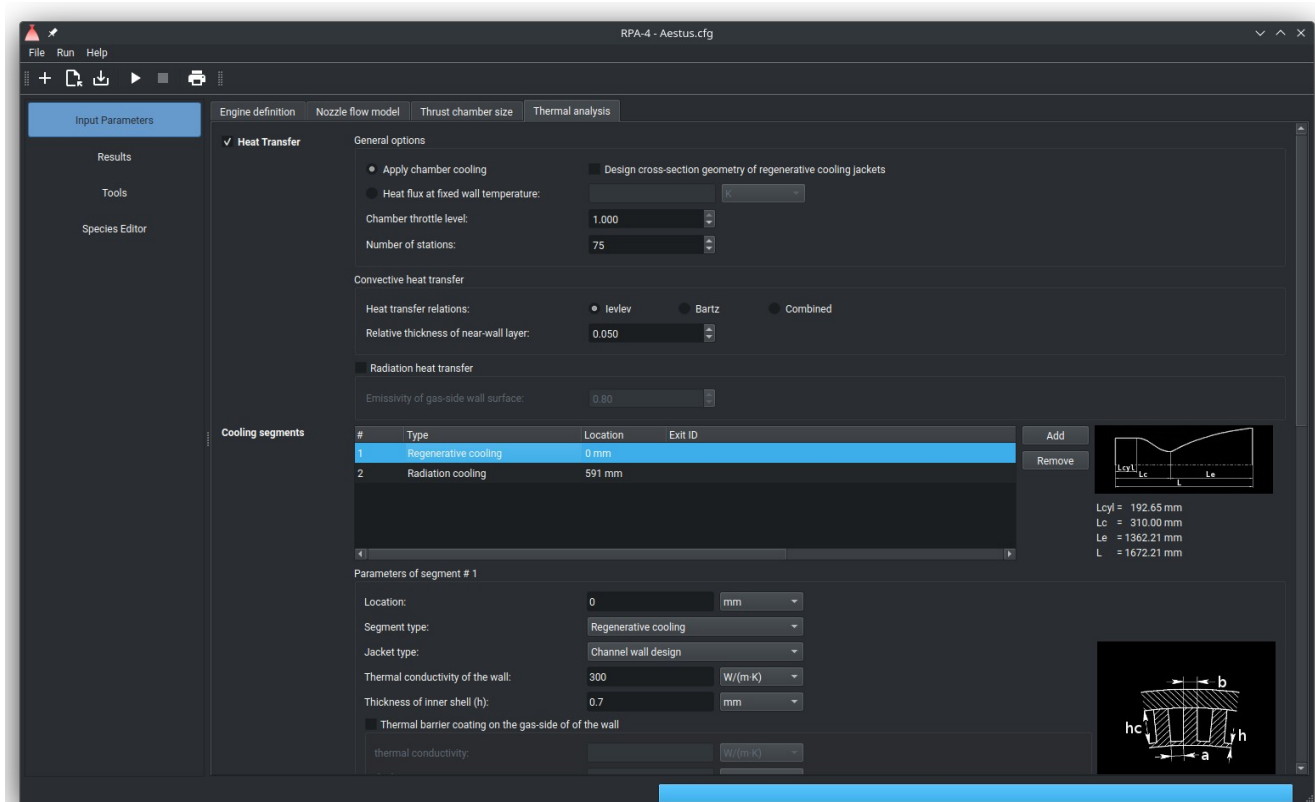
- Conical nozzle contour
- Parabolic nozzle contour
- Truncated ideal contour (TIC) calculated by axisymmetric method of characteristics (MOC)

A simplified drawing with labeled dimensions on the right side helps to identify the meaning of parameters.

Thermal Analysis

On the screen *Heat Transfer Parameters* the user can define heat transfer parameters used to calculate the heat transfer rate distribution in the thrust chamber.

The input of parameters on the screen is only enabled if the flags "*Heat Transfer*" is switched on.



Heat transfer parameters

Option "*Apply chamber cooling*" triggers the estimation of the temperature and heat flux distribution by applying the specified cooling.

Option "*Heat flux at fixed wall temperature*" triggers the estimation of the heat flux distribution at a specified constant temperature of the gas-side wall surface.

The parameter "*Chamber throttle level*" defines the throttling level of the engine. This parameter affects the conditions in the combustion chamber and the mass flow rate through the cooling jackets.

The parameter "*Number of stations*" defines the number of stations evenly distributed along the thrust chamber. The program automatically inserts stations at the nozzle inlet, nozzle throat, location of film cooling slots, and location of cooling section begin/end.

The program offers two methods for calculating gas-side convective heat transfer rates:

- levlev approach.
- Bartz approach.

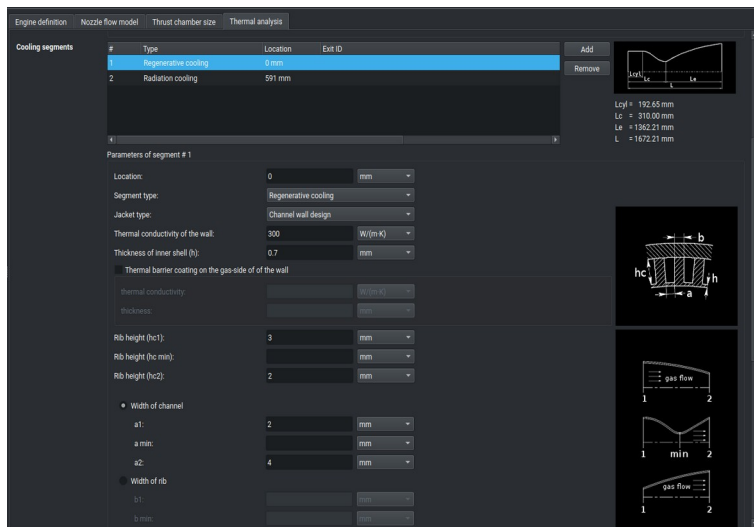
The user can choose between these two methods or a combined approach, which calculates the resulting heat transfer rate as the average value from the levlev and Bartz methods.

The program can perform thermal analysis by calculating heat flux either as convective heat transfer only or by summing the contributions of both convective and radiation heat transfers. For the latter, the user must specify a radiation emissivity coefficient for the wall's inner surface.

Cooling segments

The table *Cooling segments* contains the list of cooling segments. By clicking the row, the parameters of corresponding cooling segment are displayed below the table, available for modification.

On the right side of the table, the sketch of the thrust chamber with labeled dimensions is displayed to make the definition of cooling segments easier. Note that the dimensions are only available after the first run of the program with enabled thrust chamber sizing parameters.



Cooling segments

The user can add new segment pressing the button **Add**, or delete existing segment pressing the button **Remove** or keyboard key “Del”.

Note: new cooling segment is always created with type “Radiation cooling”. The user can change the type of the segment by clicking the created segment and changing the type in the parameters area below the table.

To view all parameters assigned to of the existing cooling segment, or to change the type of the segment or modify its parameters, click the segment on the list. All parameters of the segment will be displayed in the parameters area below the table.

The order number of the currently edited segment is displayed at the top of the parameter area.

When the parameters are modified, they are assigned to the selected segment automatically.

The user has to define the location of the each cooling segment or film cooling slot, specifying the distance from the injector plate to the beginning of the segment, or location of the film slot. The length of the cooling segment is defined implicitly by specifying the location of the next cooling segment. If the next segment is not defined, the end of the segment is defined by the nozzle exit.

The program supports 3 types of chamber cooling:

- radiation cooling
- regenerative cooling
- film cooling.

Radiation Cooling

A radiation-cooled chamber transmits the heat from the outer surface of the chamber or nozzle. Radiation cooling is typically used for small thrust chambers made of a high-temperature material, or in low-heat flux regions, such as a nozzle extension.

Configuration of the radiation-cooled section includes the following parameters:

- | | |
|---|----------|
| • location of the section | required |
| • wall thickness and thermal conductivity | required |
| • thermal barrier coating layer thickness and thermal conductivity (see also Thermal Barrier Coating Layer) | optional |
| • radiation emissivity coefficient of the wall outer surface | required |

Parameters of segment # 2

Location: 591 mm

Segment type: Radiation cooling

Thermal conductivity of the wall: 50 W/(m K)

Wall thickness: 1.5 mm

☒ Thermal barrier coating on the gas-side of the wall

thermal conductivity: W/(m K)

thickness: mm

Emissivity of outer surface of the wall: 0.80

Radiation cooling parameters

The radiation-cooled segment can be combined with a film cooling.

Regenerative Cooling

Regenerative cooling is the most widely used method for cooling a thrust chamber. This

method involves flowing high-velocity coolant over the back side of the chamber's hot gas wall to convectively cool the hot liner. The coolant, after absorbing heat from cooling the liner, is typically discharged into the injector and utilized as a propellant.

Configuration of the regenerative cooling segment includes the following parameters:

- | | |
|--|---|
| • location of the section | required |
| • type of design | required |
| • outlet/exit ID | required if the exit from the segment is hydraulically connected with the input of another regenerative cooling segment |
| • wall thickness and thermal conductivity | required |
| • thermal barrier coating layer thickness and thermal conductivity (see also <i>Thermal Barrier Coating Layer</i>) | optional |
| • coolant definition:
component/s, initial temperature and pressure, relative mass flow rate
or
connection with exit of another regenerative cooling segment, specified by its ID | required |
| • flag of opposite flow direction | optional |
| • flag of two-pass coolant flow | optional |
| • wall roughness height / step | optional |

The regenerative-cooled segment can be combined with a film cooling.

The program supports following types of regenerative cooling design:

- coaxial shell jacket design
- tubular wall jacket design
- channel wall jacket design.

Coolant definition

The list of the coolant components contains one or more species. To add species to the list, click the button **Add** at the bottom of the corresponding list. To remove the selected species from the list, click the button **Remove**.

• Coolant definition:

Component	Oxygen balance	% Mass fraction
CH6N2(L)	-173.635726	1.000000

Add
 Remove

Sum of all the mass fractions: 0

Inlet pressure: 18 bar

Inlet temperature: 298 K

Relative mass flow rate: 32.80 % (relative to the chamber mass flow rate)

☐ Coolant from segment with ID:

☒ Coolant flow direction is opposite the hot gas flow direction

☐ Two pass coolant flow

Outlet/exit ID:

Note: only use the components available in the configuration files *properties.inp* or *usr_properties.inp*, because the extended NASA database does not include all required properties such as viscosity, thermal conductivity, density, temperature of vaporization/decomposition and heat of vaporization of liquid components.

Alternatively, the user can configure the connection between the inlet of the current segment with the exit of another segment by specifying its ID. In this case, the coolant components and mass flow rate will be the same as those defined for the first segment. The initial temperature and pressure for the current segment will be equal to the temperature and pressure at the exit of the first segment, as calculated during the thermal analysis.

Coaxial Shell Jacket Design

Design-specific configuration of the segment includes the following parameters:

- distance between inner and outer walls Required

Parameters of segment # 1

Location: 0 mm

Segment type: Regenerative cooling

Jacket type: Coaxial shells

Thermal conductivity of the wall: 300 W/(m·K)

Thickness of inner shell (h): 0.7 mm

☐ Thermal barrier coating on the gas-side of the wall

thermal conductivity: W/(m·K)

thickness: mm

Distance between shells (hc): 3 mm

Tube diameter (d min): mm

Tube diameter (d2): 2 mm

• Width of channel

a1: 2 mm

a min: mm

a2: 4 mm

• Width of rib

b1: mm

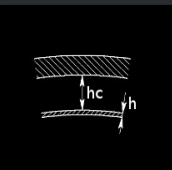
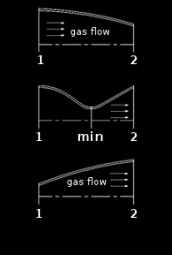
b min: mm

b2: mm

Number of tubes: 120

☐ Helix

☐ Wall roughness height / step: mm

Coaxial shell jacket parameters

The program supports segments with a constant distance between the walls. To simulate a chamber with a variable inter-wall distance, define multiple segments, each with a different inter-wall distance.

Typical issues with configuration:

- too small inter-wall distance, or too large coolant mass flow rate leads to a high pressure drop; the message "Negative pressure drop" appears;
- due to too large inter-wall distance or too small mass flow rate, the program cannot converge the solution.

Possible solution: change one or several relevant parameters.

Tubular Wall Jacket Design

Design-specific configuration of the segment includes the following parameters:

- diameter of tube “d1”, “d2” at both sides of the segment required
- diameter of tube “d min” at minimal diameter of the segment optional; can be defined if a nozzle throat is located within the current segment
- tube wall thickness “h” and thermal conductivity required
- number of tubes required
- flag of helical tube wall jacket design required

Parameters of segment # 1

Location: 0 mm

Segment type: Regenerative cooling

Jacket type: Tubular design

Thermal conductivity of the wall: 300 W/(m·K)

Thickness of inner shell (h): 0.7 mm

☐ Thermal barrier coating on the gas-side of the wall

thermal conductivity: W/(m·K)

thickness: mm

Tube diameter (d1): 3 mm

Tube diameter (d min): mm

Tube diameter (d2): 2 mm

• Width of channel

a1: 2 mm

a min: mm

a2: 4 mm

• Width of rib

b1: mm

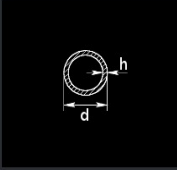
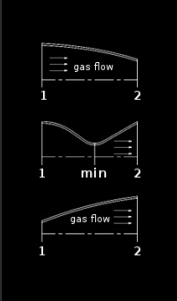
b min: mm

b2: mm

Number of tubes: 120

☐ Helix degrees

☐ Wall roughness height / step: mm

Tube wall jacket parameters

The program supports segments with a constant number and variable diameter of the tubes. To simulate a chamber with a variable number of tubes, define multiple segments, each with different corresponding parameters.

Typical issues with configuration:

- inappropriate number or diameter of tubes, or too high coolant mass flow rate leads to a high pressure drop; the message "Negative pressure drop" appears;
- due to inappropriate number or diameter of tubes, or too small mass flow rate the program cannot converge the solution.

Possible solution: change one or several relevant parameters, or replace the single cooling segment with two or more segments with optimum parameters.

Channel Wall Jacket Design

Design-specific configuration of the segment includes the following parameters:

- | | |
|--|---|
| • height of ribs "hc1", "hc2" at both sides of the segment | required |
| • height of ribs "hc min" at minimal diameter of the segment | optional; can be defined if a nozzle throat is located within the current segment |
| • width of channels "a1", "a2" at both sides of the segment | required |
| • width of channels "a min" at minimal diameter of the segment | optional; can be defined if a nozzle throat is |

- number of channels “ n ”
or
width of ribs “ $b1$ ”, “ $b2$ ” at both sides of the segment
- width of ribs “ b_{min} ” at minimal diameter of the segment
- helix angle

located within the
current segment
required

optional; can be defined
if a nozzle throat is
located within the
current segment

optional

Note: index "1" corresponds to the parameters at the section end which is closer to injector plate; index "2" corresponds to the parameters at the section end which is closer to nozzle exit.

Parameters of segment # 1

Location: 0 mm

Segment type: Regenerative cooling

Jacket type: Channel wall design

Thermal conductivity of the wall: 300 W/(m·K)

Thickness of inner shell (h): 0.7 mm

☐ Thermal barrier coating on the gas-side of the wall

thermal conductivity: W/(m·K)

thickness: mm

Rib height (hc1): 3 mm

Rib height (hc min): mm

Rib height (hc2): 2 mm

☒ Width of channel

a1: 2 mm

a min: mm

a2: 4 mm

☐ Width of rib

b1: mm

b min: mm

b2: mm

Number of channels: 120

☐ Helix angle: degrees

☐ Wall roughness height / step: mm

Channel wall jacket parameters

The program supports segments with a constant number of channels, and helix angle. To simulate a chamber with a variable number of channels and/or angle, define multiple segments, each with different corresponding parameters.

Typical issues with configuration:

- inappropriate geometry and/or number of channels, or too high coolant mass flow rate leads to a high pressure drop; the message "Negative pressure drop" appears;
- due to inappropriate geometry and/or number of channels, or too small mass flow rate

the program cannot converge the solution.

Possible solution: change one or several relevant parameters or replace the single cooling segment with two or more segments with optimum parameters.

Film Cooling

Film cooling provides protection from an excessive heat by introducing a thin film of the coolant through orifices around the injector periphery or in the chamber wall. This method is typically used in a high heat flux regions in combination with regenerative or radiation cooling.

Configuration of the film cooling includes the following parameters:

- location (the distance from the injector plate) required
- coolant definition: required
 - component/s,
 - initial temperature,
 - relative mass flow rate

Parameters of segment # 2

Location: 591 mm

Segment type: Film cooling

Coolant definition:

Component	Oxygen balance	% Mass fraction
H2(L)	-793.668274	1.000000

Sum of all the mass fractions: 1.000000

Relative mass flow rate: 2.00 % (relative to the chamber mass flow rate)

Film cooling parameters

Both liquid and gaseous species can be used as a film coolants.

Thrust chamber with multiple cooling segments

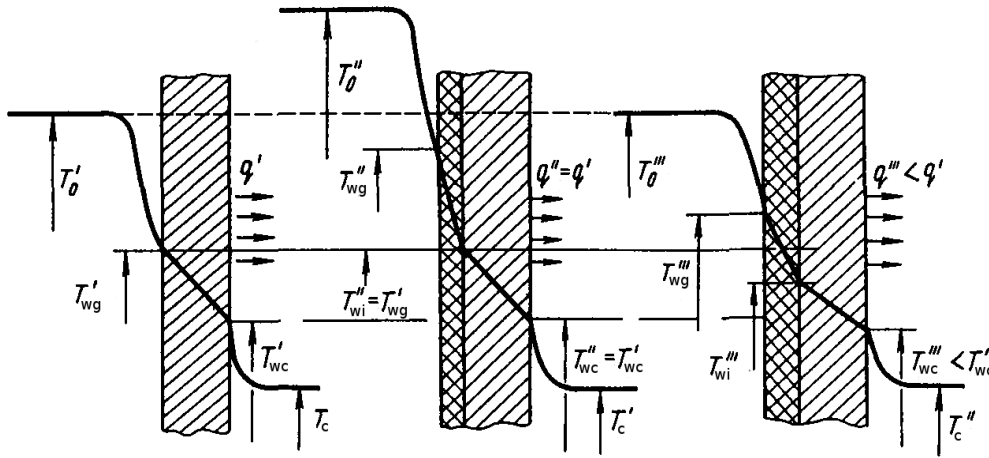
For one chamber, multiple cooling segments can be defined

- to combine different types of cooling (e.g. regeneratively cooled combustion chamber + radiation cooled nozzle extension + one or more film cooling);
- to optimize parameters along the chamber (e.g. number of tubes/channels, tube/channel geometry etc).

Thermal Barrier Coating Layer

The thermal barrier coating (TBC) is used to reduce the temperature gradient across the chamber wall.

The functional principle of a TBC, illustrated in figure below, is to increase the hot gas side wall temperature T_{wg} . This is achieved by applying a ceramic top layer (e.g. Y2O3-stabilized ZrO2, PYSZ) onto the wall base material. The PYSZ ceramic offers a thermal conductivity λ of about 1.5 W/(m K). In order to prevent a coating overhear (>1500 K) coating thicknesses of less than 0.1 mm are required. These high hot gas side wall temperatures compared to the maximum temperatures of the uncoated wall lead to the remarkable reduction in heat flux.



Functional principle of a TBC

With such a protective layer, the application range of existing regenerative or radiation cooling thrust chambers can be enlarged towards higher combustion chamber pressures or an increased thrust chamber life.

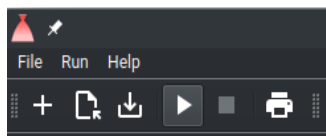
In general a TBC acts as an insulator to the inner liner, reducing the wall heat flux into the wall base material.

To analyze the thermal state of the thrust chambers with TBC consisting of several layers with known thicknesses $\delta_1, \delta_2, \dots, \delta_N$ and thermal conductivities $\lambda_1, \lambda_2, \dots, \lambda_N$, the effective thermal conductivity λ of the TBC with the thickness $\delta_1 + \delta_2 + \dots + \delta_N$ can be calculated using the following relation:

$$\lambda = \frac{\delta_1 + \delta_2 + \dots + \delta_N}{\frac{\delta_1}{\lambda_1} + \frac{\delta_2}{\lambda_2} + \dots + \frac{\delta_N}{\lambda_N}}$$

Starting Analysis

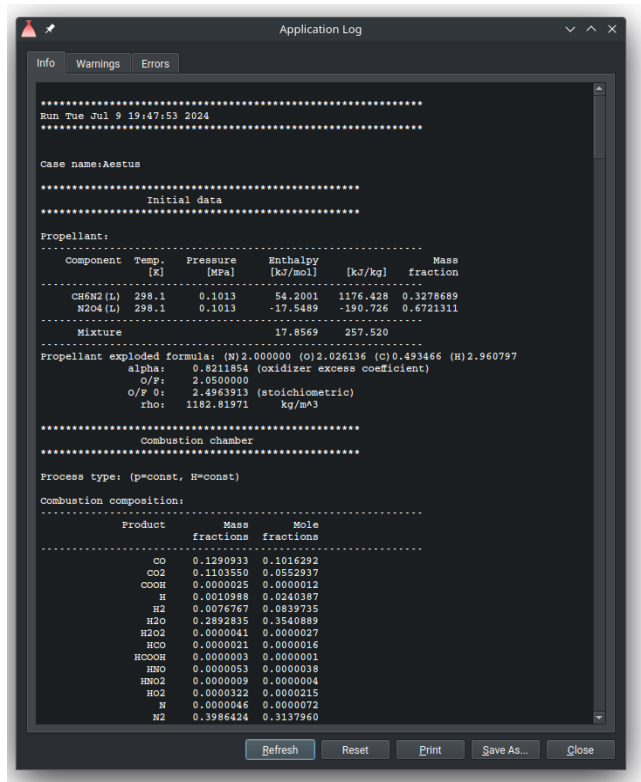
The analysis is started by clicking menu **Run**, and then **Start analysis** in menu bar, or clicking the icon **Run** on the main toolbar:



After completing the analysis, the program automatically switches to the screen *Results*.

To print out the results, use the item **Run** in a menu bar, and then item **Print results**.

To check the analysis log, use the item **Run** in a menu bar, and then item **Show log**. The analysis log in ASCII format is also available as a file in the user directory.



Analysis Log

Results

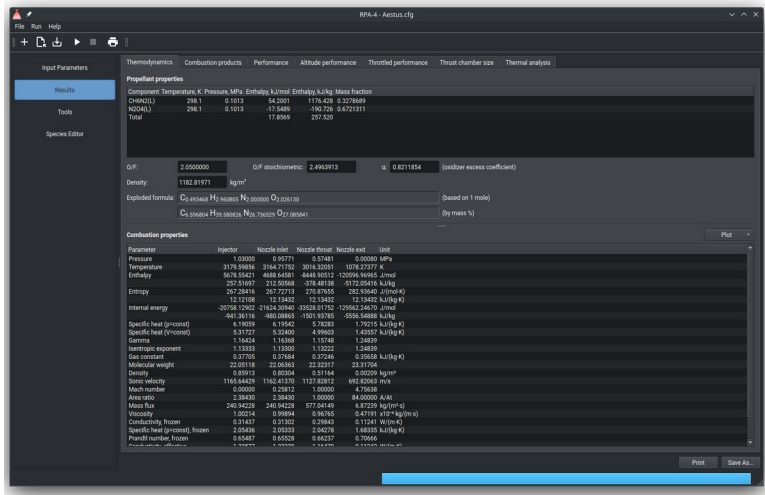
The screen *Results* consists of following tabs: *Thermodynamics*, *Combustion products*, *Performance*, *Altitude performance*, *Throttled performance*, *Thrust chamber size*, and *Thermal analysis*.

To print out the results, use the button **Print**. To save the results as PDF or HTML file, use the button **Save As...** Both buttons are placed at the right-bottom corner of the screen.

Thermodynamics

The tab *Thermodynamics* displays the properties of the propellant and the reaction products

at the chamber stations involved in the analysis.



Thermodynamic properties

The following properties of the propellant are available:

- propellant composition with initial parameters of components
- propellant mixture ratio (O/F)
- propellant stoichiometric mixture ratio (O/F),
- oxidizer excess coefficient,
- propellant density (only if density values of all components are available),
- propellant exploded formula.

Propellant properties					
Component	Temperature, K	Pressure, MPa	Enthalpy, kJ/mol	Enthalpy, kJ/kg	Mass fraction
CH6N2(L)	298.1	0.1013	54.2001	1176.428	0.3278689
N2O4(L)	298.1	0.1013	-17.5489	-190.726	0.6721311
Total			17.8569	257.520	

O/F:	2.0500000	O/F stoichiometric:	2.4963913	α:	0.8211854 (oxidizer excess coefficient)
Density:	1182.81971	kg/m ³			
Exploded formula:	C _{0.493468} H _{2.960805} N _{2.000000} O _{2.026130}				(based on 1 mole)
	C _{6.596804} H _{39.580826} N _{26.736529} O _{27.085841}				(by mass %)

Propellant properties

If the problem is configured to solve the parameters in the combustion chamber (see chapter *Nozzle Flow Model*), the only available nozzle station is *Injector* (see figure *Injector thermodynamic properties*), otherwise the available chamber stations are *Injector*, *Nozzle inlet*, *Nozzle throat* and *Nozzle exit* as well as additional stations, defined on screen *Nozzle Flow Model* (see figure *Thermodynamic properties above*).

Rocket Propulsion Analysis v.4.0.0

Parameter	Injector	Nozzle inlet	Nozzle throat	Nozzle exit	Unit
Pressure	1.03000	0.95771	0.57481	0.00080	MPa
Temperature	3179.59856	3164.71752	3016.32051	1078.27377	K
Enthalpy	5678.55421	4688.64581	-8448.90512	-120596.96965	J/mol
	257.51697	212.50568	-378.48138	-5172.05416	kJ/kg
Entropy	267.28416	267.72713	270.87655	282.93640	J/(mol·K)
	12.12108	12.13432	12.13432	12.13432	kJ/(kg·K)
Internal energy	-20758.12902	-21624.30940	-33528.01752	-129562.24670	J/mol
	-941.36116	-980.08865	-1501.93785	-5556.54888	kJ/kg
Specific heat (p=const)	6.19059	6.19542	5.78283	1.79215	kJ/(kg·K)
Specific heat (V=const)	5.31727	5.32400	4.99603	1.43557	kJ/(kg·K)
Gamma	1.16424	1.16368	1.15748	1.24839	
Isentropic exponent	1.13333	1.13300	1.13222	1.24839	
Gas constant	0.37705	0.37684	0.37246	0.35658	kJ/(kg·K)
Molecular weight	22.05118	22.06363	22.32317	23.31704	
Density	0.85913	0.80304	0.51164	0.00209	kg/m³
Sonic velocity	1165.64429	1162.41370	1127.82812	692.82063	m/s
Mach number	0.00000	0.25812	1.00000	4.75638	
Area ratio	2.38430	2.38430	1.00000	84.00000	A/At
Mass flux	240.94228	240.94228	577.04149	6.87239	kg/(m²·s)
Viscosity	1.00214	0.99894	0.96765	0.47191	$\times 10^{-4}$ kg/(m·s)
Conductivity, frozen	0.31437	0.31302	0.29843	0.11241	W/(m·K)
Specific heat (p=const), frozen	2.05436	2.05333	2.04278	1.68335	kJ/(kg·K)
Prandtl number, frozen	0.65487	0.65528	0.66237	0.70666	
Conductivity, effective	1.00214	0.99894	0.96765	0.47191	$\times 10^{-4}$ W/(m·K)

Combustion properties

If the problem is configured to calculate the performance for the combustion chamber with infinite cross section area, the parameters for *Nozzle inlet* station are identical to that at *Injector* station.

The thermodynamic parameters are displayed in the current units, defined in the *Preferences Dialog*.

Combustion products

Screen *Combustion products* displays the table with products of reaction at each nozzle station.

Rocket Propulsion Analysis v.4.0.0

Thermodynamics

Combustion products

Performance

Altitude performance

Throttled performance

Thrust chamber size

Thermal analysis

Combustion products

Product	Injector mass fraction	Injector mole fraction	Nozzle inlet mass fraction	Nozzle inlet mole fraction	Nozzle throat mass fraction	Nozzle throat mole fraction	Nozzle exit mass fraction	Nozzle exit mole fraction
CO	0.1290933	0.1016292	0.1287091	0.1013839	0.1223414	0.0975018	0.0430055	0.0357998
CO2	0.1103550	0.0552937	0.1109592	0.0556277	0.1209666	0.0613582	0.2456226	0.1301347
COOH	0.0000025	0.0000012	0.0000023	0.0000011	0.0000014	0.0000007		
H	0.0010988	0.0240387	0.0010917	0.0238974	0.0008815	0.0195223		
H2	0.0076767	0.0839735	0.0076518	0.0837482	0.0071589	0.0792747	0.0097316	0.1125615
H2O	0.2892835	0.3540889	0.2897614	0.3548740	0.2993318	0.3709074	0.2976482	0.3852417
H2O2	0.0000041	0.0000027	0.0000039	0.0000025	0.0000022	0.0000015		
HCO	0.0000021	0.0000016	0.0000020	0.0000015	0.0000011	0.0000008		
HCOOH	0.0000003	0.0000001	0.0000003	0.0000001	0.0000002	0.0000001		
HNCO	0.0000002	0.0000001	0.0000002	0.0000001				
HNO	0.0000053	0.0000038	0.0000050	0.0000036	0.0000027	0.0000020		
HNO2	0.0000009	0.0000004	0.0000008	0.0000004	0.0000004	0.0000002		
HO2	0.0000322	0.0000215	0.0000307	0.0000206	0.0000178	0.0000120		
N	0.0000046	0.0000072	0.0000044	0.0000069	0.0000023	0.0000036		
N2	0.3986424	0.3137960	0.3987479	0.3140561	0.4000737	0.3188070	0.4039920	0.3362621
N2O	0.0000020	0.0000010	0.0000018	0.0000009	0.0000011	0.0000005		
NH	0.0000017	0.0000025	0.0000016	0.0000023	0.0000008	0.0000012		
NH2	0.0000008	0.0000011	0.0000007	0.0000010	0.0000004	0.0000005		
NH3	0.0000007	0.0000009	0.0000006	0.0000008	0.0000004	0.0000005		
NO	0.0114314	0.0084008	0.0112070	0.0082405	0.0083795	0.0062339		
NO2	0.0000066	0.0000032	0.0000063	0.0000030	0.0000036	0.0000017		
O	0.0047596	0.0065599	0.0046963	0.0064763	0.0032984	0.0046021		
O2	0.0157018	0.0108205	0.0155824	0.0107443	0.0121931	0.0085062		
OH	0.0318932	0.0413515	0.0315323	0.0409066	0.0253407	0.0332610		
Gaseous fraction:	1.0000000	1.0000000	1.0000000	1.0000000	1.0000000	1.0000000	1.0000000	1.0000000
Condensed fraction:	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000	0.0000000

Mass fraction low limit: 1.000000e-07

Apply

Mass fraction low limit: 1.000000e-07

Apply

Combustion properties

Performance

The screen *Performance* displays the theoretical (ideal) performance of the chamber, as well as its estimated delivered performance and the correction factors used to calculate the delivered performance.

Rocket Propulsion Analysis v.4.0.0

Thermodynamics

Combustion products

Performance

Altitude performance

Throttled performance

Thrust chamber size

Thermal analysis

Ideal performance

Parameter	Sea level	Optimum expansion	Vacuum	Units
Characteristic velocity		2260.62696		m/s
Effective exhaust velocity	3173.06304	4187.12650	4350.25218	m/s
Specific impulse (by mass)	3173.06304	4187.12650	4350.25218	N·s/kg
Specific impulse (by weight)	323.56238	426.96808	443.60227	s
Thrust coefficient	1.40362	1.85220	1.92436	
Thrust	1023.76646	1350.94690	1403.57825	kN
Altitude	0.00000	14.05808		km
Ambient pressure	1.00000	0.13857	0.00000	atm

Estimated performance

Parameter	Sea level	Optimum expansion	Vacuum	Units
Characteristic velocity		2239.13669		m/s
Effective exhaust velocity	3034.89715	4048.96061	4212.08629	m/s
Specific impulse (by mass)	3034.89715	4048.96061	4212.08629	N·s/kg
Specific impulse (by weight)	309.47338	412.87908	429.51327	s
Thrust coefficient	1.35539	1.80827	1.88112	
Thrust	979.18821	1306.36865	1359.00000	kN
Altitude	0.00000	14.05808		km
Ambient pressure	1.00000	0.13857	0.00000	atm

Reaction efficiency:

0.99049

Nozzle efficiency:

0.97753

Overall efficiency:

0.96824

Chamber performance

If the problem is configured to solve the combustion parameters at *Injector* station (see chapter *Nozzle Flow Model*), the performance parameters are not available, otherwise the tab displays calculated performance of the chamber at following ambient conditions:

- sea level (SL) conditions ($p_a=1 \text{ atm}$, or 14.7 psi)
- optimum nozzle expansion ($p_e=p_a$)
- vacuum conditions ($p_a=0$).

In case if a flow separation in the nozzle is predicted, the table contains an additional column with the performance at the sea level with respect to the flow separation.

Rocket Propulsion Analysis v.4.0.0

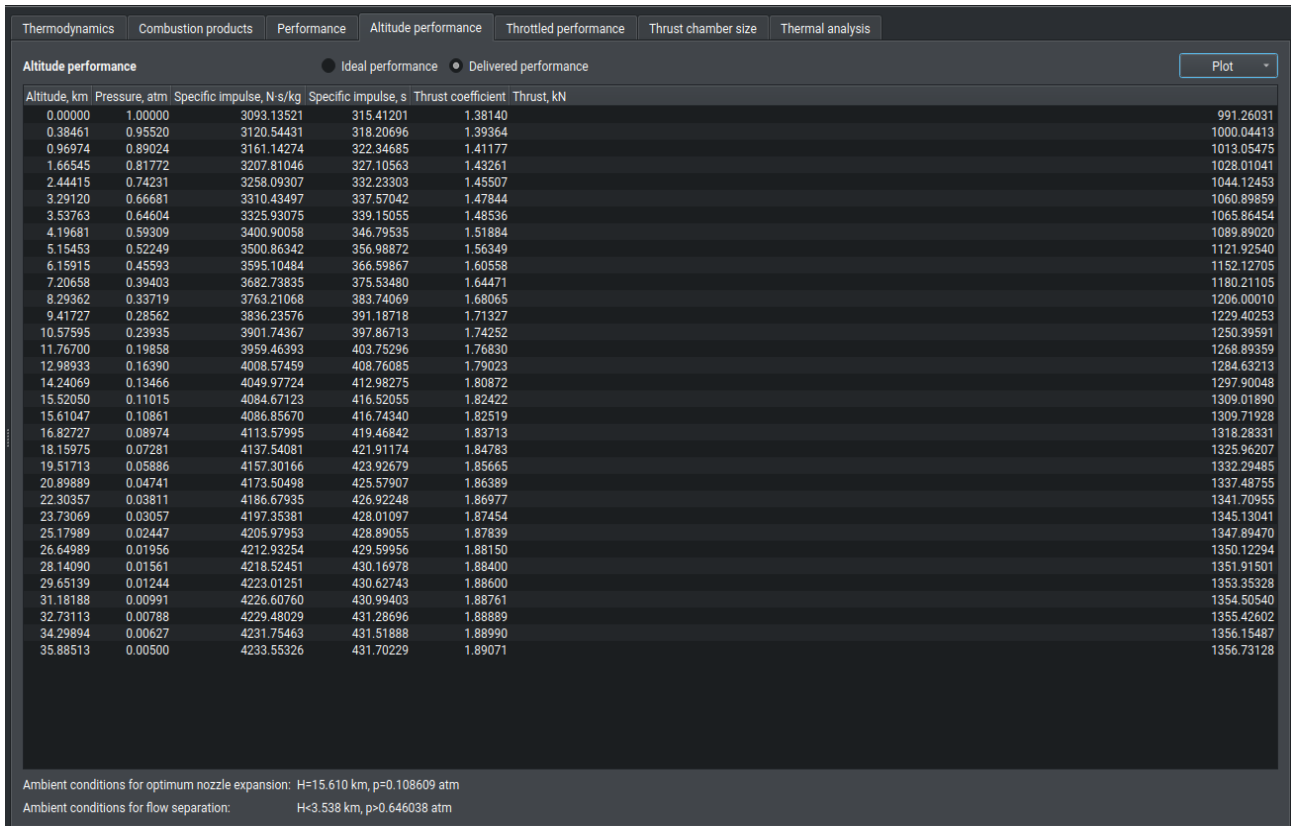
Thermodynamics	Combustion products	Performance	Altitude performance	Throttled performance	Thrust chamber size	Thermal analysis
Ideal performance						
Parameter	Sea level	Sea level (flow sep.)	Flow separation	Optimum expansion	Vacuum	Units
Characteristic velocity				2260.62696		m/s
Effective exhaust velocity	2596.27032	3129.98110	3546.11616	4274.81584	4416.66589	m/s
Specific impulse (by mass)	2596.27032	3129.98110	3546.11616	4274.81584	4416.66589	N-s/kg
Specific impulse (by weight)	264.74589	319.16925	361.60321	435.90990	450.37458	s
Thrust coefficient	1.14847	1.38456	1.56864	1.89099	1.95373	
Thrust	824.96034	994.54601	1126.77219	1358.31524	1403.38784	kN
Altitude	0.00000	0.00000	5.81021	17.72713		km
Ambient pressure	1.00000	1.00000	0.47822	0.07792	0.00000	atm
Estimated performance						
Parameter	Sea level	Sea level (flow sep.)	Flow separation	Optimum expansion	Vacuum	Units
Characteristic velocity				2239.13669		m/s
Effective exhaust velocity	2456.57531	2993.35822	3406.42116	4135.12083	4276.97088	m/s
Specific impulse (by mass)	2456.57531	2993.35822	3406.42116	4135.12083	4276.97088	N-s/kg
Specific impulse (by weight)	250.50097	305.23759	347.35829	421.66498	436.12966	s
Thrust coefficient	1.09711	1.33684	1.52131	1.84675	1.91010	
Thrust	780.57250	951.13433	1082.38435	1313.92739	1359.00000	kN
Altitude	0.00000	0.00000	5.81021	17.72713		km
Ambient pressure	1.00000	1.00000	0.47822	0.07792	0.00000	atm
Reaction efficiency: 0.99049 Nozzle efficiency: 0.97766 Overall efficiency: 0.96837						

Chamber performance with flow separation

Altitude performance

The tab *Altitude performance* displays the performance of the thrust chamber at the configured ambient conditions.

Rocket Propulsion Analysis v.4.0.0



Altitude performance

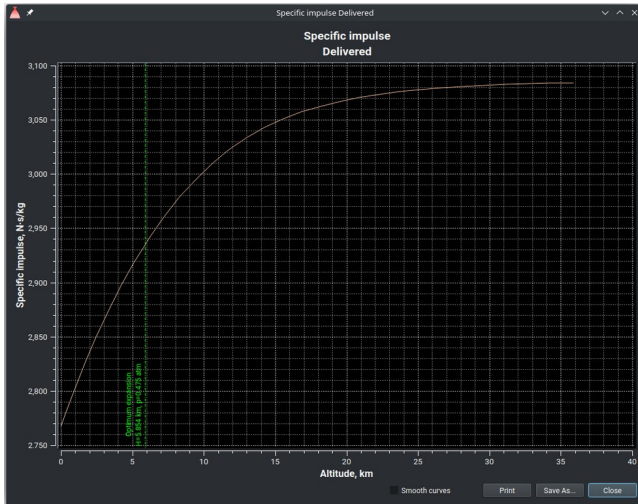
The code displays the results both for the ideal theoretical performance and the estimated delivered performance.

To plot the diagrams "*specific impulse vs. altitude*", "*thrust coefficients vs. altitude*" or "*thrust vs. altitude*", click the button **Plot** at the upper-right corner of the tab.

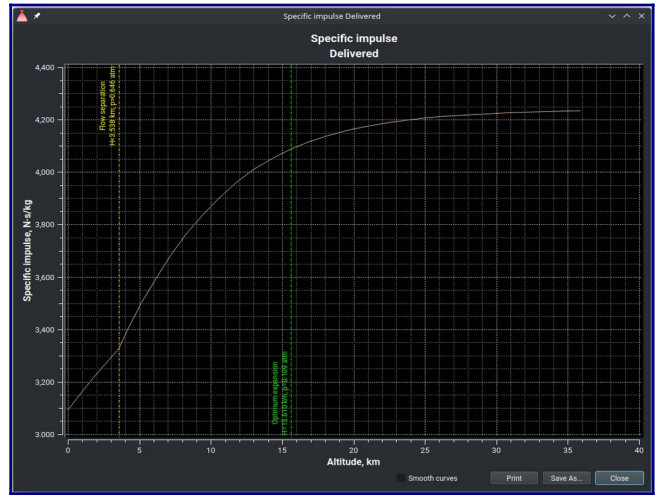
The altitude of an optimum expansion, as well as the altitude of a flow separation (if predicted) are shown on the diagram by corresponding vertical marker line/s.

The performance parameters are displayed in the current units, defined in the *Preferences Dialog*.

Rocket Propulsion Analysis v.4.0.0



Plot - altitude performance



Plot - altitude performance with flow separation

Throttled performance

The tab *Throttled performance* displays the performance of the thrust chamber at the configured throttling values.

Thermodynamics		Combustion products		Performance		Altitude performance		Throttled performance		Thrust chamber size		Thermal analysis	
Throttled chamber performance												Plot	
Throttle value	Mass flow rate, kg/s	Pressure, MPa	Ideal performance		Delivered performance		Thrust, kN	Specific impulse, N/s/kg		Thrust, kN		Specific impulse, N/s/kg	
			Sea level	Sea level	Sea level	Optimum expansion		Optimum expansion	Optimum expansion	Vacuum	Vacuum	Vacuum	
0.30000	169.84600	4.65612	2100.21218	214.16204	356.71265	2873.23759	292.98870	488.00792	3021.02850	308.05917	513.1		
0.34000	192.49214	5.28357	2161.96503	220.45908	416.16127	2876.50710	293.32209	553.70500	3024.77374	308.44108	582.2		
0.38000	215.13827	5.91169	2213.12859	225.67631	476.12866	2880.03557	293.68190	619.60587	3028.54924	308.82608	651.5		
0.40111	227.09133	6.24347	2237.28122	228.13919	508.06717	2881.21829	293.80250	654.29969	3029.98660	308.97265	688.0		
0.42000	237.78440	6.54042	2273.29803	231.81189	540.55482	2882.51985	293.93522	685.41827	3031.42545	309.11937	720.8		
0.46000	260.43054	7.16968	2342.04707	238.82234	609.94058	2885.07615	294.19589	751.36194	3034.25119	309.40751	790.2		
0.50000	283.07667	7.79942	2399.99009	244.73088	679.38121	2887.39899	294.43276	817.35530	3036.81859	309.66932	859.6		
0.54000	305.72281	8.42961	2449.51214	249.78072	748.87173	2889.52551	294.64960	883.39385	3039.16877	309.90897	929.1		
0.58000	328.36894	9.06020	2492.34268	254.14822	818.40793	2891.48477	294.84939	949.47379	3041.33385	310.12974	998.6		
0.62000	351.01507	9.69117	2529.76659	257.96440	887.98621	2893.29987	295.03448	1015.59187	3043.33943	310.33426	1068.2		
0.66000	373.66121	10.32248	2562.75858	261.32865	957.60346	2894.98950	295.20677	1081.74527	3045.20619	310.52461	1137.8		
0.70000	396.30734	10.95412	2592.07154	264.31774	1027.25698	2896.56899	295.36784	1147.93156	3046.95108	310.70254	1207.5		
0.74000	418.95348	11.58607	2618.29639	266.99193	1096.94437	2898.05104	295.51896	1214.14856	3048.58816	310.86948	1277.2		
0.78000	441.59961	12.21830	2641.90343	269.39918	1166.66352	2899.44633	295.66124	1280.39436	3050.12925	311.02662	1346.9		
0.82000	464.24574	12.85079	2663.27170	271.57813	1236.41255	2900.76386	295.79559	1346.66727	3051.58432	311.17500	1416.6		
0.86000	486.89188	13.48355	2682.71011	273.56030	1306.18976	2902.01136	295.92280	1412.96576	3052.96191	311.31548	1486.4		
0.90000	509.53801	14.11654	2700.47298	275.37161	1375.93963	2903.19545	296.04355	1479.28843	3054.26936	311.44880	1556.2		
0.94000	532.18414	14.74976	2716.77161	277.03361	1445.82277	2904.32188	296.15841	1545.63406	3055.51302	311.57562	1626.0		
0.98000	554.83028	15.38320	2731.78304	278.56435	1515.67594	2905.39567	296.26791	1612.00149	3056.69845	311.69650	1695.9		
1.00000	566.15334	15.70000	2738.85368	279.28535	1550.61117	2905.91421	296.32078	1645.19305	3057.27087	311.75487	1730.8		
1.02000	577.47641	16.01685	2745.65670	279.97907	1585.55198	2906.42120	296.37248	1678.38969	3057.83051	311.81193	1765.8		
1.06000	600.12255	16.65070	2758.51966	281.29072	1655.44984	2907.40236	296.47253	1744.79770	3058.91348	311.92237	1835.7		
1.10000	622.76868	17.28473	2770.48062	282.51040	1725.36856	2908.34257	296.56841	1811.22466	3059.95116	312.02818	1905.6		
1.14000	645.41481	17.91895	2781.63313	283.64764	1795.30723	2909.24490	296.66042	1877.66976	3060.94695	312.12972	1975.5		
1.18000	668.06095	18.55334	2792.05818	284.71070	1865.26503	2910.11208	296.74885	1944.13223	3061.90386	312.22730	2045.5		
1.22000	690.70708	19.18790	2801.82621	285.70676	1935.24120	2910.94656	296.83394	2010.61140	3062.82461	312.32119	2115.5		
1.26000	713.35321	19.82263	2810.99880	286.64210	2005.23503	2911.75056	296.91593	2077.10662	3063.71164	312.41164	2185.5		
1.30000	735.99935	20.45751	2819.63000	287.52224	2075.24584	2912.52606	296.99500	2143.61728	3064.56716	312.49888	2255.5		
1.34000	758.64548	21.09254	2827.76747	288.35203	2145.27302	2913.27487	297.07136	2210.14282	3065.39318	312.58311	2325.5		
1.38000	781.29162	21.72771	2835.45342	289.13578	2215.31598	2913.99865	297.14517	2276.68272	3066.19151	312.66452	2395.5		
1.42000	803.93775	22.36303	2842.72530	289.87731	2285.37418	2914.69890	297.21657	2343.23647	3066.96382	312.74327	2465.6		
1.46000	826.58388	22.99848	2849.61655	290.58002	2355.44711	2915.37698	297.28572	2409.80362	3067.71162	312.81953	2535.7		
1.50000	849.23002	23.63407	2856.15703	291.24696	2425.53428	2916.03415	297.35273	2476.38374	3068.43631	312.89343	2605.8		

Throttled performance

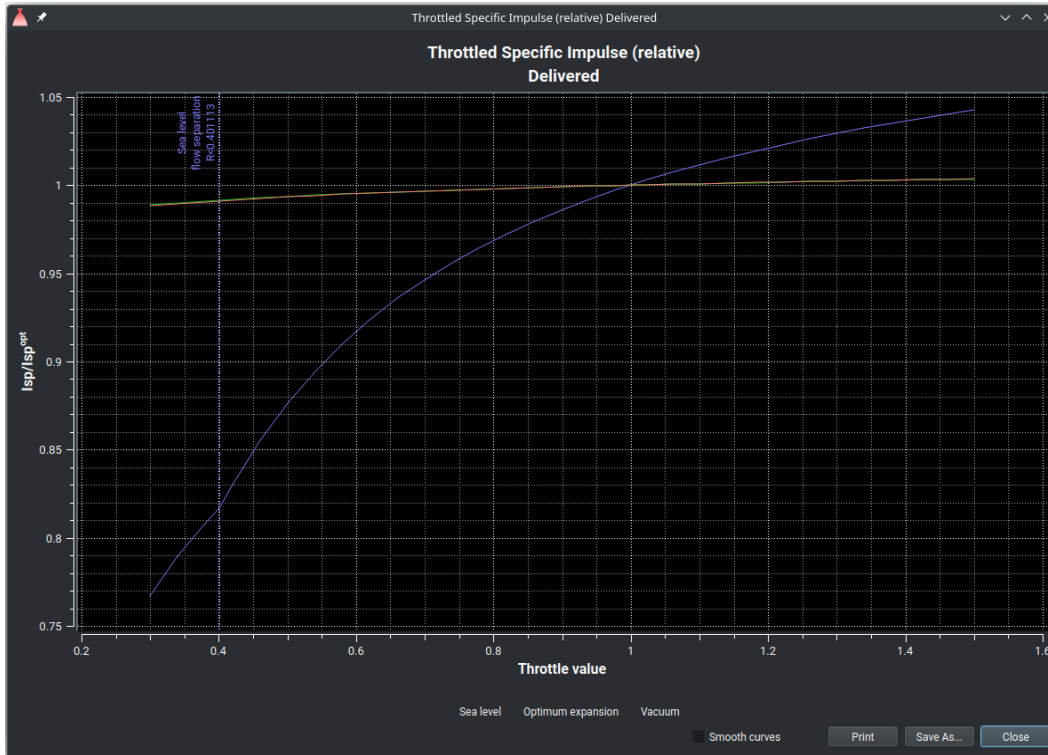
The code display the results both for the ideal theoretical performance and the estimated

delivered performance.

To plot the diagrams, click the button **Plot** at the upper-right corner of the tab.

The throttling value with a predicted flow separation is shown on the diagram by vertical marker line.

The performance parameters are displayed in the current units, defined in the *Preferences Dialog*.

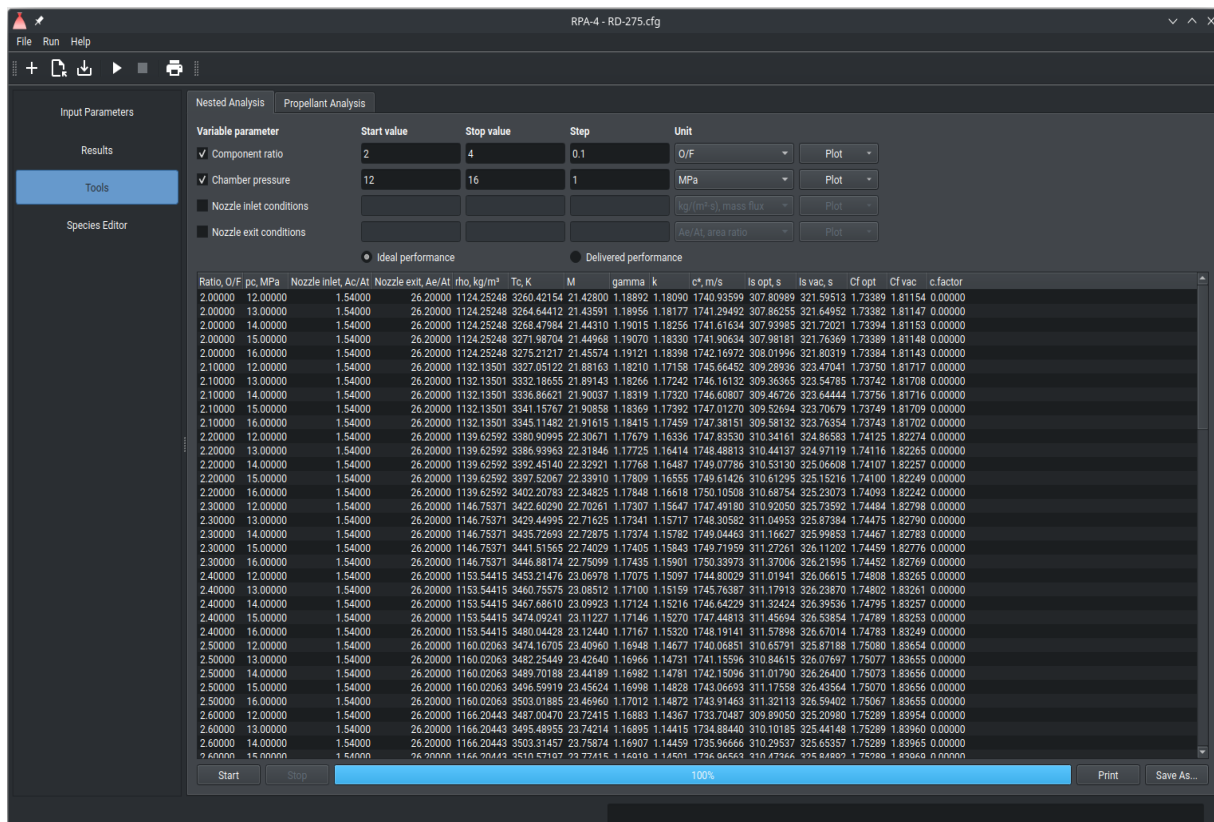


Plot - throttled performance

Nested Analysis

Using the nested analysis, the user can evaluate the performance of the thrust chamber across a range of parameters, with up to four independent variables: component ratio, chamber pressure, nozzle inlet conditions, and nozzle exit conditions.

Rocket Propulsion Analysis v.4.0.0



Nested analysis

Nested analysis inherits the initial configuration parameters defined on the screens *Engine Definition*, *Propellant Specification*, and *Nozzle Flow Model*. The user may define up to four variables parameters to replace the corresponding parameters in the initial configuration. To start the nested analysis, click the **Start** button at the bottom-left corner of the tab.

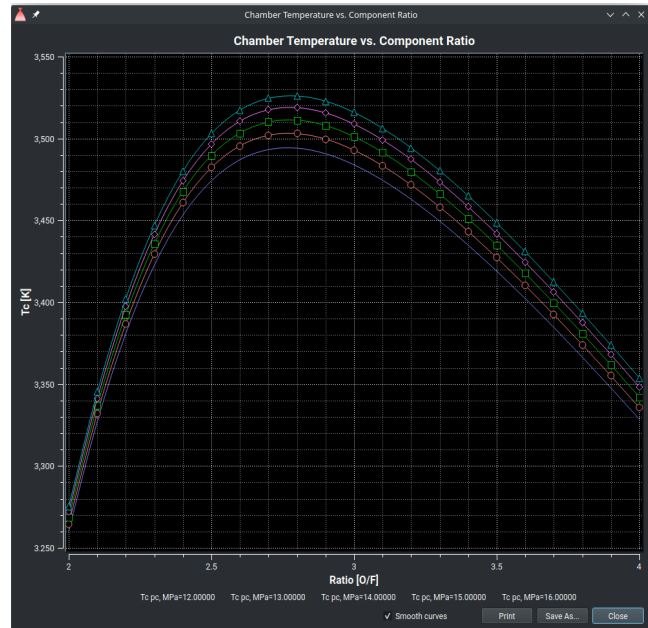
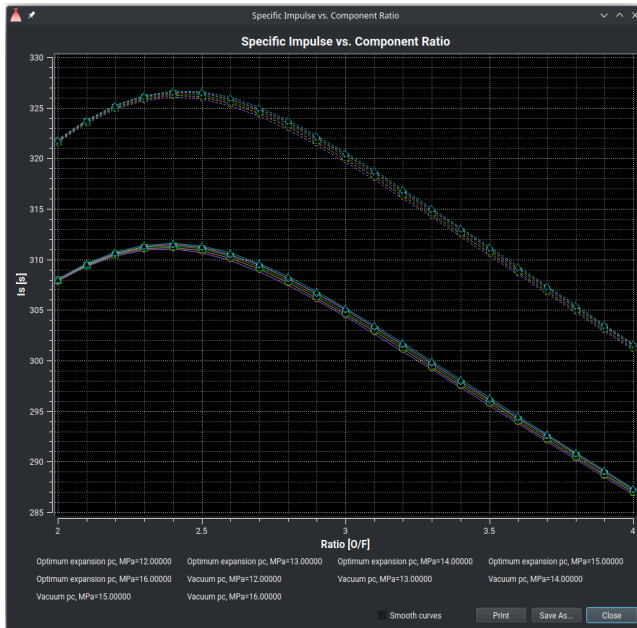
Note: since the program performs the thermodynamic analysis for each unique combination of variable parameters, using small step size can result in very long execution time.

The code displays the results both for the ideal theoretical performance and the estimated delivered performance.

The program performs thermodynamic analysis for each unique combination of variable parameters, stepping it by defined value, and displaying the results in a table at the bottom of the tab. The progress of the nested analysis is indicated by a progress bar at the bottom of the tab. To stop the running analysis, click the **Stop** button.

After finishing the nested analysis, the user can plot the diagrams "*specific impulse vs. variable parameter*", "*chamber temperature vs. variable parameter*", "*characteristic velocity vs. variable parameter*" or "*thrust coefficient vs. variable parameter*" by clicking the button **Plot** at the right side of the corresponding parameter configuration.

Rocket Propulsion Analysis v.4.0.0



Nested analysis plots

To print out the results of the nested analysis, click the button **Print**. To save the results as ASCII, HTML, or PDF file, click the button **Save As...** Both buttons are located at the right-bottom corner of the screen.

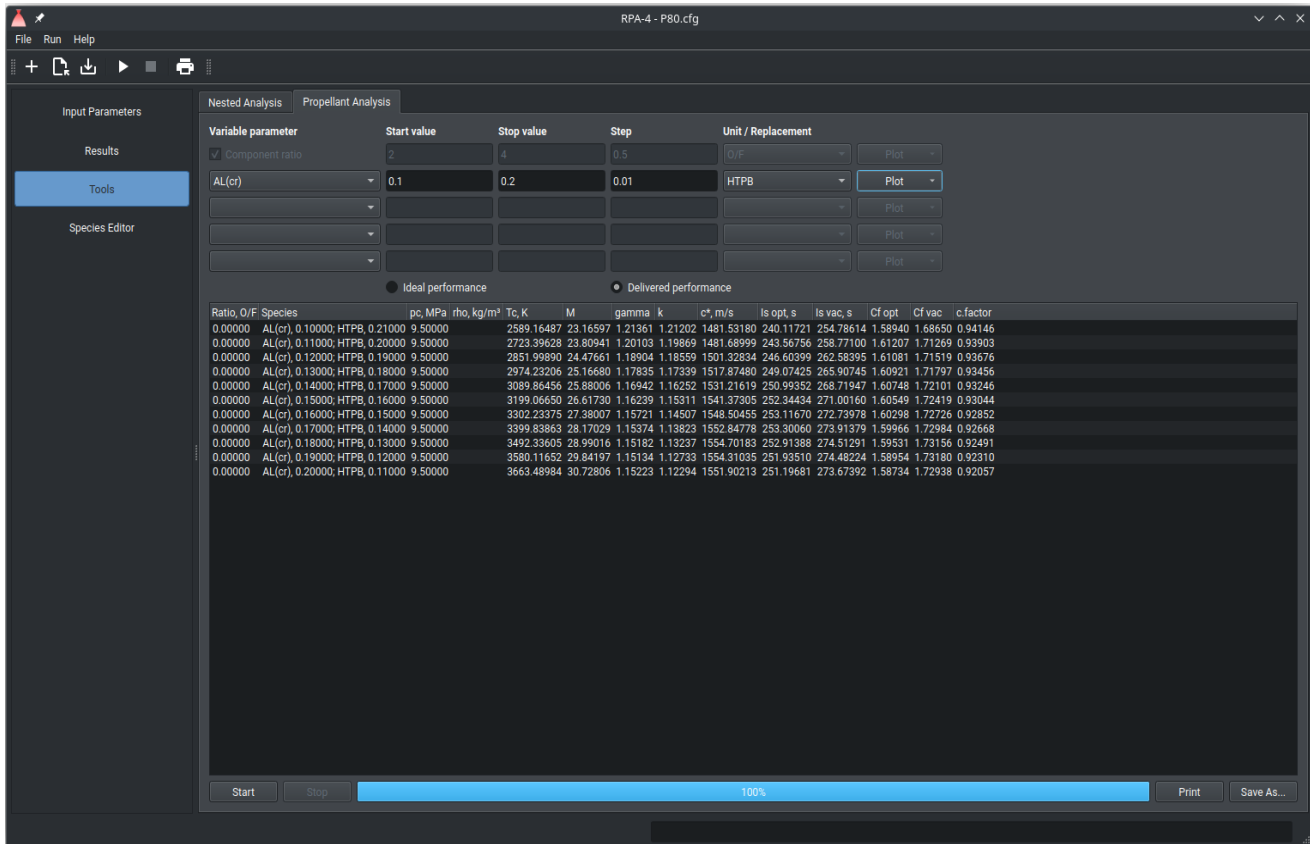
The defined parameters of the nested analysis are stored in the INI-file and shared between different configurations for the same user.

The defined parameters are not stored in the current configuration file.

Propellant Analysis

Using the propellant analysis tool, the user can evaluate different propellant compositions by sequentially replacing one component with another.

Rocket Propulsion Analysis v.4.0.0



Propellant analysis

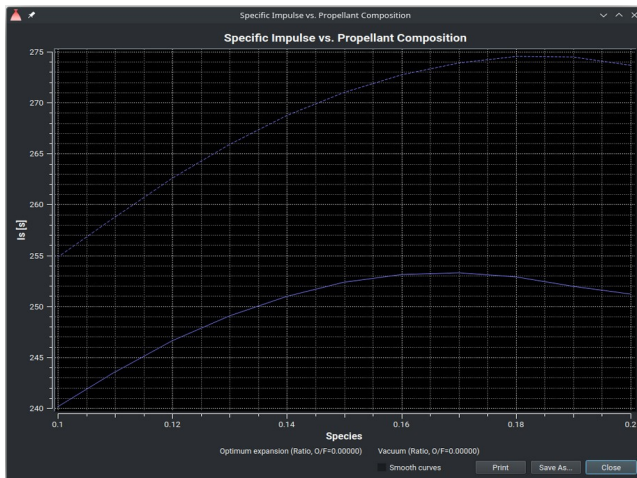
Propellant analysis inherits the initial configuration parameters defined on the screens *Engine Definition*, *Propellant Specification*, and *Nozzle Flow Model*. The user can define up to four pairs of components; within each pair, one component replaces the corresponding counterpart component in the initial propellant composition. To start the propellant analysis, click the button **Start** at the left-bottom corner of the tab.

Note: since the program performs the thermodynamic analysis for each unique combination of variable parameters, using small step size can result in very long execution time.

The code displays the results both for the ideal theoretical performance and the estimated delivered performance.

The program performs thermodynamic analysis for each unique combination of variable parameters, stepping it by defined value, and displaying the results in a table at the bottom of the tab. The progress of the propellant analysis is indicated by a progress bar at the bottom of the tab. To stop the running analysis, click the **Stop** button.

After finishing the propellant analysis, the user can plot the diagrams "*specific impulse vs. variable parameter*", "*chamber temperature vs. variable parameter*", "*characteristic velocity vs. variable parameter*" or "*thrust coefficient vs. variable parameter*" by clicking the button **Plot** at the right side of the corresponding parameter configuration.



Propellant analysis plot

To print out the results of the nested analysis, click the button **Print**. To save the results as ASCII, HTML, or PDF file, click the button **Save As...** Both buttons are located at the right-bottom corner of the screen.

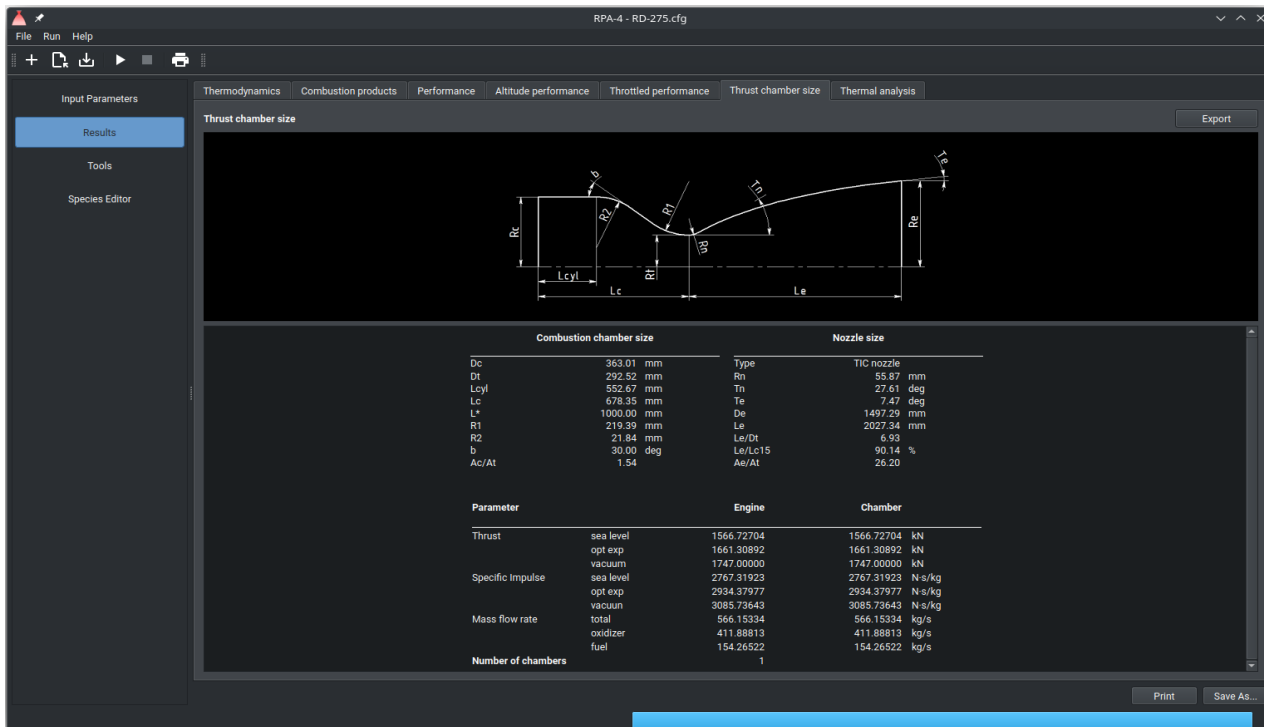
The defined parameters of the nested analysis are stored in the INI-file and shared between different configurations for the same user.

The defined parameters are not stored in the current configuration file.

Thrust chamber size

The tab *Size and Geometry* displays the calculated size of the thrust chamber.

Rocket Propulsion Analysis v.4.0.0

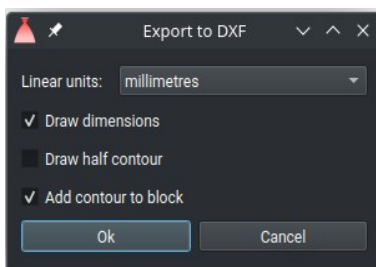


Size and geometry

After the initial program run, the user can use the calculated values for divergence thrust loss and friction thrust loss (from the results of *Thermal Analysis*) to adjust the nozzle efficiency on the *Nozzle Shape and Efficiencies* screen.

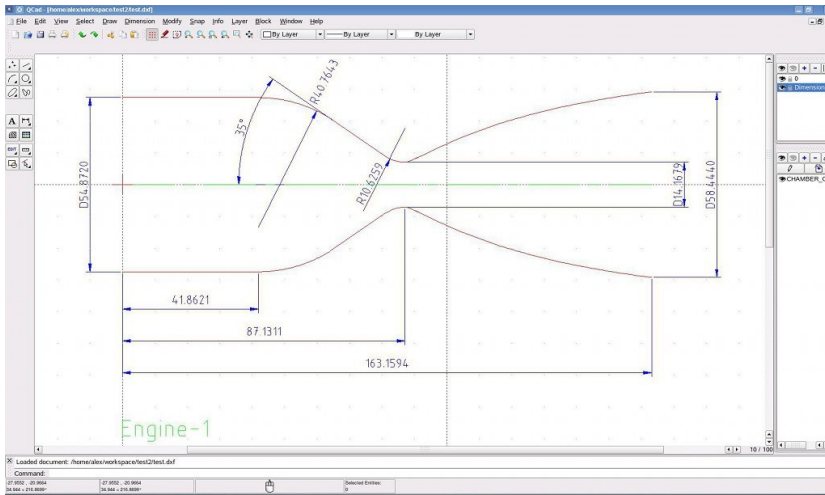
To save the results as an ASCII file, click the button **Save As...** To export the contour to the DXF file, click the button **Export to DXF** at the right-bottom corner of the screen.

When exporting to DXF file, additional export parameters have to be provided:



The resulting file can be opened in any CAD software that supports the DXF format.

Rocket Propulsion Analysis v.4.0.0



DXF file opened in QCad

Thermal Analysis

File

Run

Help

+

↶

↷

⏮

⏭

⏪

⏩

🖨

Input Parameters

Results

Tools

Species Editor

RPA-4 - Aestus.cfg

Thermodynamics

Combustion products

Performance

Altitude performance

Throttled performance

Thrust chamber size

Thermal analysis

Thermal analysis

Location, mm

Radius, mm

Conv heat coe
kW/m²·K

Conv heat flux,
kW/m²

Rad heat flux,
kW/m²

Total heat flux,
kW/m²

Twg, K

Twl, K

Twc, K

Tc, K

pc, MPa

wc, m/s

p, kg/m²

N

Helix angle,
degrees

A, m²

0.00

105.00

1.06345

2987.66971

0.00000

2987.66971

552.72916

552.72916

545.49204

422.23339

1.73385

4.71327

840.00000

120

0.00000

0.

12.04

105.00

1.06361

2990.95421

0.00000

2990.95421

550.07681

550.07681

542.82619

419.43236

1.73508

4.71327

840.00000

120

0.00000

0.

24.08

105.00

1.06378

2994.24495

0.00000

2994.24495

547.42171

547.42171

540.15752

416.62826

1.73632

4.71327

840.00000

120

0.00000

0.

36.12

105.00

1.06394

2997.54197

0.00000

2997.54197

544.76383

544.76383

537.48662

413.82107

1.73755

4.71327

840.00000

120

0.00000

0.

48.16

105.00

1.06411

3000.84528

0.00000

3000.84528

542.10319

542.10319

534.81170

411.01080

1.73879

4.71327

840.00000

120

0.00000

0.

60.20

105.00

1.06428

3004.15490

0.00000

3004.15490

539.43976

539.43976

532.13455

408.19744

1.74002

4.71327

840.00000

120

0.00000

0.

72.24

105.00

1.06445

3007.47086

0.00000

3007.47086

536.77356

536.77356

529.45456

405.38099

1.74126

4.71327

840.00000

120

0.00000

0.

84.28

105.00

1.06462

3010.79318

0.00000

3010.79318

534.10456

534.10456

526.77172

402.56143

1.74249

4.71327

840.00000

120

0.00000

0.

96.33

105.00

1.06479

3014.12187

0.00000

3014.12187

531.43278

531.43278

524.08604

399.73876

1.74373

4.71327

840.00000

120

0.00000

0.

108.37

105.00

1.06496

3017.45687

0.00000

3017.45687

528.75819

528.75819

521.37950

396.91298

1.74496

4.71327

840.00000

120

0.00000

0.

120.41

105.00

1.06513

3020.79848

0.00000

3020.79848

526.08081

526.08081

518.70511

394.08408

1.74620

4.71327

840.00000

120

0.00000

0.

132.45

105.00

1.06531

3024.14645

0.00000

3024.14645

523.40062

523.40062

516.01185

391.25206

1.74743

4.71327

840.00000

120

0.00000

0.

144.49

105.00

1.06548

3027.50088

0.00000

3027.50088

520.71762

520.71762

513.31473

388.41690

1.74867

4.71327

840.00000

120

0.00000

0.

156.53

105.00

1.06566

3030.86180

0.00000

3030.86180

518.03181

518.03181

510.61473

385.57860

1.74990

4.71327

840.00000

120

0.00000

0.

168.57

105.00

1.06584

3034.22924

0.00000

3034.22924

515.34317

515.34317

507.91185

382.73716

1.75113

4.71327

840.00000

120

0.00000

0.

180.61

105.00

1.06601

3037.60321

0.00000

3037.60321

512.65171

512.65171

505.20609

379.89257

1.75237

4.71327

840.00000

120

0.00000

0.

192.65

105.00

1.06619

3040.98374

0.00000

3040.98374

509.95741

509.95741

502.46650

377.04474

1.75360

4.71327

840.00000

120

0.00000

0.

204.68

104.53

1.07595

3071.25824

0.00000

3071.25824

507.69188

507.69188

500.15581

374.21171

1.75483

4.72485

840.00000

120

0.00000

0.

216.51

103.09

1.10583

3158.15647

0.00000

3158.15647

506.24013

506.24013

498.49784

371.33230

1.75609

4.75837

840.00000

120

0.00000

0.

228.44

100.67

1.15901

3310.70069

0.00000

3310.70069

505.65410

505.65410

497.55456

368.35933

1.75740

4.81776

840.00000

120

0.00000

0.

240.37

97.22

1.24171

3546.43292

0.00000

3546.43292

506.06598

506.06598

497.41752

365.23651

1.75879

4.90712

840.00000

120

0.00000

0.

252.30

92.65

1.36509

3896.55558

0.00000

3896.55558

507.72394

507.72394

498.26270

361.89138

1.76033

5.03412

840.00000

120

0.00000

0.

264.22

86.84

1.54915

4416.22233

0.00000

4416.22233

511.05879

511.05879

508.39224

358.22278

1.76207

5.21141

840.00000

120

0.00000

0.

276.16

79.54

1.83177

5211.88639

0.00000

5211.88639

516.87418

516.87418

506.66650

354.08469

1.76415

5.40692

840.00000

120

0.00000

0.

288.09

72.61

2.18053

6189.45902

0.00000

6189.45902

523.63446

523.63446

508.86183

349.70750

1.76653

5.74069

840.00000

120

0.00000

0.

300.02

68.92

2.38606

6769.50118

0.00000

6769.50118

525.04828

525.04828

508.92718

345.46455

1.76903

5.90446

840.00000

120

0.00000

0.

312.00

68.00

2.41536

6861.48420

0.00000

6861.48420

521.38395

521.38395

505.04275

342.00027

1.77115

5.94753

840.00000

120

0.00000

0.

338.86

78.17

1.69568

4881.43437

0.00000

4881.43437

483.40650

483.40650

471.60629

334.09800

1.77584

5.51566

840.00000

120

0.00000

0.

357.72

93.88

1.09357

3186.64730

0.00000

3186.64730

488.16778

488.16778

440.22924

326.76761

1.77987

4.99870

840.00000

120

0.00000

0.

381.58

110.06

0.72663

2164.77943

0.00000

2164.77943

423.39928

423.39928

417.22445

320.93786

1.78397

4.51816

840.00000

120

0.00000

0.

405.44

126.27

0.51509

1522.93327

0.00000

1522.93327

405.50165

405.50165

401.28894

316.26751

1.78551

4.24066

851.30232

120

0.00000

0.

429.30

142.26

0.37426

1112.15785

0.00000

1112.15785

390.55743

390.55743

387.26844

312.47995

1.78771

3.98654

857.27299

120

0.00000

0.

453.16

157.95

0.28219

842.10417

0.00000

842.10417

377.99364

377.99364

375.29346

309.35457

1.78969

3.80375

860.17088

120

0.00000

0.

477.03

173.15

0.22074

660.93719

0.00000

660.93719

367.91932

367.91932

365.60676

306.72113

1.79154

3.66756

862.67074

120

0.00000

0.

500.89

187.94

0.17784

533.96033

0.00000

533.96033

359.62921

359.62921

357.58206

304.45437

1.79332

3.56702

864.70404

120

0.00000

0.

524.75

202.35

0.14562

438.25956

0.00000

438.25956

352.27351

352.27351

350.42671

302.47570

1.79507

3.49500

866.55300

120

0.00000

0.

548.61

216.26

0.12314

372.24286

0.00000

372.24286

346.59074

346.59074

344.58204

300.71742

1.79682

3.44400

868.45098

120

0.00000

0.

572.47

229.85

0.10565

319.15249

0.00000

319.15249

341.32797

341.32797

339.70944

299.13057

1.79859

3.41947

869.62901

120

0.00000

0.

591.00

240.13

0.09462

286.17708

0.00000

286.17708

337.68309

337.68309

336.12629

298.00000

1.80000

3.41041

870.67500

120

0.00000

0.

591.00

240.13

0.08086

160.48645

0.00000

160.48645

337.37642

337.37642

337.14630

596.93

240.93

0.07881

155.48736

0.00000

155.48736

336.09354

336.09354

336.09354

Plot

Print

Save As...

Thermal analysis results

Screen *Thermal Analysis* displays the results of thermal analysis of thrust chamber in one table.

The thrust chamber geometry is printed out in columns "*Location*" and "*Radius*".

The columns "*Conv.heat flux*", "*Rad.heat flux*", and "*Heat flux*" display the calculated heat flux: convection heat flux, radiation heat flux, and the sum of convection heat flux and radiation heat flux at the corresponding locations.

The column "*T_{wg}*" displays the temperature of chamber wall on its hot gas side.

The column "*T_{wi}*" displays the temperature between the thermal barrier coating layer and chamber wall (if coating is available) or the temperature of chamber wall on its hot gas side (the same as *T_{wg}*).

The column "*T_{wc}*" displays the temperature of chamber wall on its cooler side.

The columns "*p_c*", "*T_c*", "*wc*", and "*ρ*" display the pressure, temperature, velocity, and density of the coolant correspondingly. The column "*N*" displays the number of channels or tubes.

The table contains also design parameters for cooling segments.

Using context menu, the content of the complete table or selected rows can be copied into the clipboard.

To save the results as ASCII, HTML, pr PDF file, click the button "**Save As...**" at the right-bottom corner of the screen.

To plot the diagrams "*Heat Flux vs. Location*" or "*Temperatures vs. Location*", click the button **Plot** at the bottom-right corner of the tab. The location of the nozzle throat is shown on the diagram by corresponding vertical marker line.

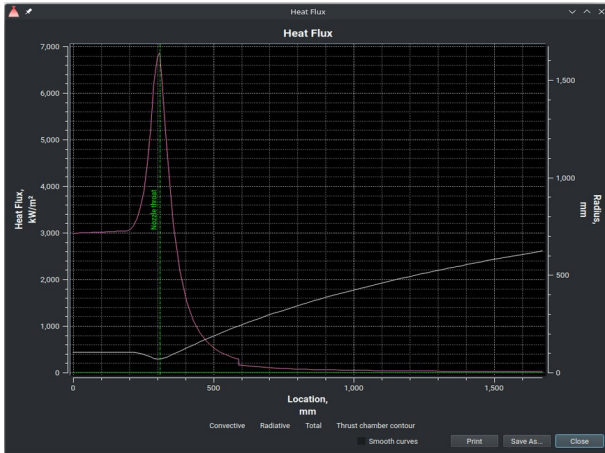


Diagram „Heat flux vs. Location“

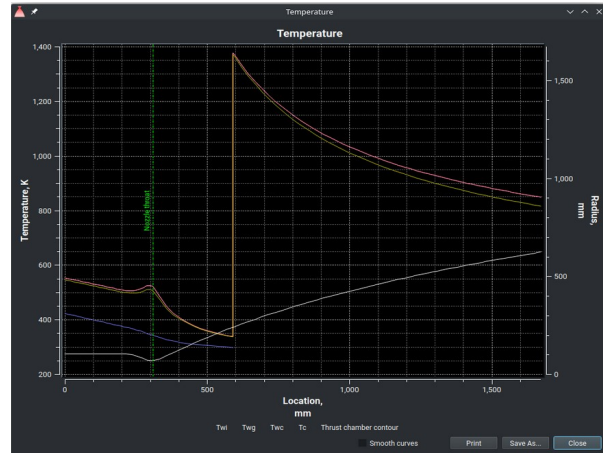


Diagram „Temperatures vs. Location“

All results are displayed in the current units, defined in the Preferences Dialog.

Propellant Feed System (Cycle Analysis)

The version 4.0.0 of the program does not contain the module “Propellant Feed System”.

Thermodynamic Database Editor

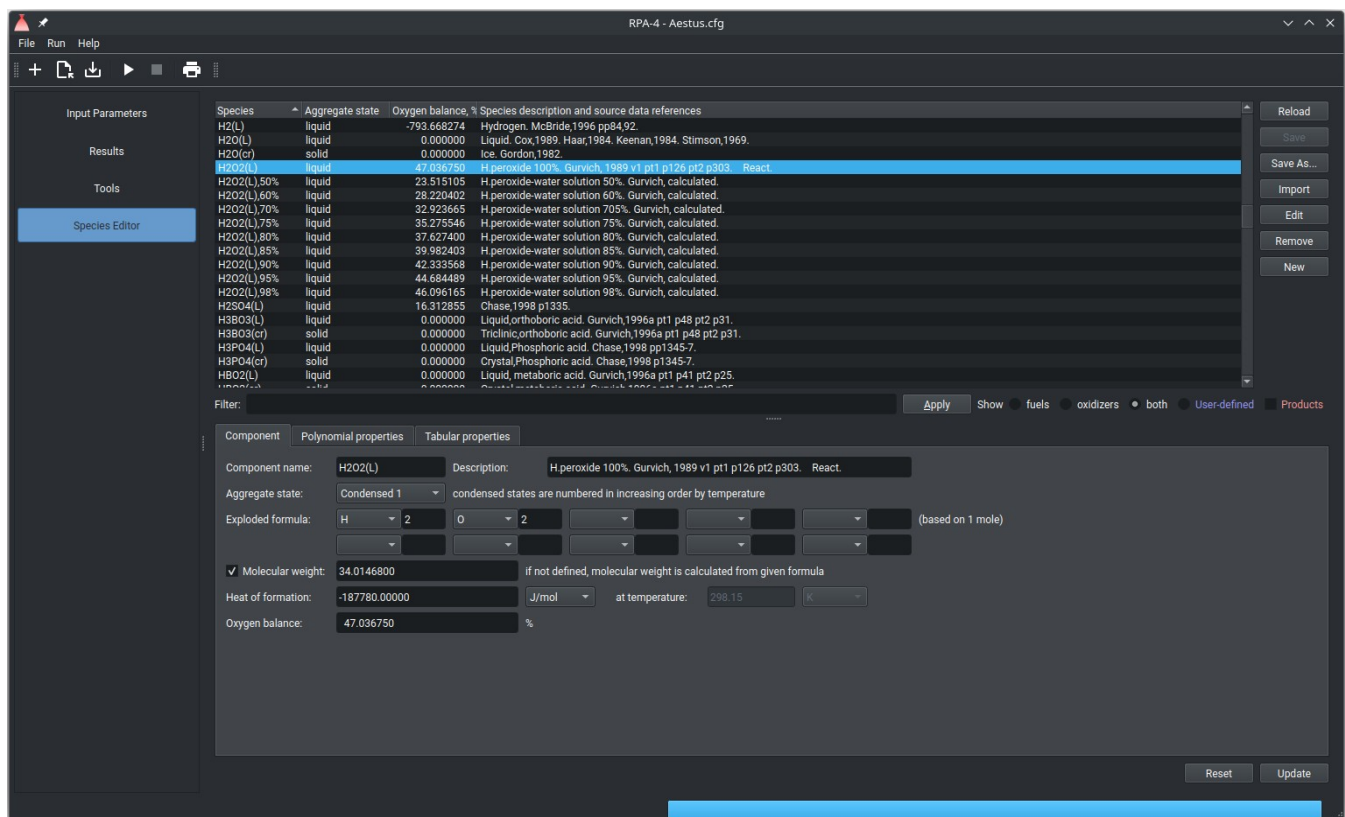
The Thermodynamic Database Editor is an integrated species viewer and editor. This tool allows users to easily define new propellant components or import components from PROPEP or CEA2 species databases.

RPA distribution package contains two database files `resources/thermo.inp` and `resources/properties.inp`.

The file `resources/thermo.inp` contains the thermodynamic properties in format described in the reference documentation available on NASA web site <https://www1.grc.nasa.gov/research-and-engineering/ceaweb/>.

Users can define custom species and save them in two additional database files `resources/usr_thermo.inp` and `resources/usr_properties.inp`. These files are not included in the standard RPA distribution packages and will not be overwritten during program updates.

The tool features a database viewer at the top of the screen and a species editor at the bottom. The views can be resized by dragging the horizontal bar between the viewer and the editor.



Thermodynamic Database Editor

The viewer includes a species table, filter options, and control buttons. The species table

displays available species according to the filter settings.

- To display only fuels, only oxidizers, or both fuels and oxidizers, select the corresponding radio buttons.
- To view all species, including atomized and/or ionized reaction products, check the **All species** box. Leave it unchecked to see only possible propellant components

The filter pattern is applied to two columns “*Species*” and “*Species description and source data references*”.

The following functions are assigned to control buttons:

- To reload the database from the files, click the button **Reload**. All changes performed since the database was last time loaded or saved will be lost.
- To save the changes made since the database was last time loaded or saved, click the button **Save**. The program creates a backup copy of the database files, adding the character “~” to the name of the file.
- To save the current database in specified location, click the button **Save As...**
- To import the species from external file in *CEA2* or *PROPEP* formats, click the button **Import**. After successful loading the external database, the program displays the list of available species in the dialog window. Select one or more species to be imported, and click the button **OK**. All imported species are immediately available for thermodynamic analysis.
- To display the data of the selected species into the species editor, click the button **Edit**. Alternatively, double click the species on the list to display it in the editor.
- To remove the species from the loaded database, click the button **Remove**. Note that removed species are immediately unavailable for thermodynamic analysis.
- To reset the editor for creating a new species, click the button **New**.

Note: all new or updated species are saved into the user-defined database files `resources/usr_thermo.inp` and `resources/usr_properties.inp`.

Note: although the user can import any component from *PROPEP* species database, do not replace all components already available in *CEA2/RPA* database: the sources of the data in *CEA2* file are [NASA Glen thermodynamic database](#) and [Gurvich thermodynamic database](#), both known for their high accuracy.

Note: since *PROPEP* library does not contain the component's temperature, always check standard temperature and tabular data for imported components.

Note: always check the log (click item **Run** in main menu, and then **Show log**; check the tabs “*Warnings*” and/or “*Errors*”) just after the import from *PROPEP* library.

The editor consists of three tabs *Component*, *Polynomial properties*, and *Tabular properties*, and the control buttons at the bottom of the editor:

Rocket Propulsion Analysis v.4.0.0

To save changes to existing species or save new species, click the button **Update**.

To reset the species data, click the button **Reset**.

The tab *Component* displays information about component, including its aggregate state, chemical formula, molecular weight, heat of formation, and the temperature at which the heat of formation is defined.

The component name serves as the species identifier and must be unique within the database. The suffix "(L)" can be added to the name for liquid components.

The description typically contains the common name of the species and reference information.

The chemical formula is provided as either a molecular formula (if applicable) or an exploded formula, followed by its molecular weight. The heat of formation (enthalpy) can be specified in one of the available units: J/mol, cal/mol, kJ/kg, kcal/kg, Btu/lbm, kcal/lbm.

The heat of formation is followed by the temperature (specified in one of the units: K, R, C, F), at which the heat of formation is defined. Note that if polynomial properties are available, the temperature is always 298.15 K and cannot be changed.

Polynomial properties for the one or more temperature intervals are given by 9 coefficients as described in documentation for CEA2 available on NASA web site <https://www1.grc.nasa.gov/research-and-engineering/ceaweb/>. Click the button **Add** to add new temperature interval; click the button **Remove** to delete selected temperature interval.

Tabular properties for the one or more pressure and temperature intervals are specified by values of specific heat C_p (kJ/mol-K), density ρ (kg/m³) and dynamic viscosity μ (muPa-s) for each unique combination of pressure and temperature. Click the button **Add p** to add new pressure interval; click the button **Remove p** to delete selected pressure interval. Click the button **Add T** to add new temperature interval; click the button **Remove T** to delete selected temperature interval.

Note: For the components supplied with thermodynamic properties in the polynomial form, the specification of the specific heat is not required (define "0" instead).

Note: For the gaseous components, the specification of the density is not required.

In the database file, the tabular data are formatted as follows:

```
!p, MPa    T, K      Cp, kJ/mol-K  rho, kg/m3  mu, muPa-s
Comp_name  2,3
p1         T1       Cp11      rho11      mu11
```

p1	T2	Cp12	rho12	mu12
p1	T3	Cp13	rho13	mu13
p2	T1	Cp21	rho21	mu21
p2	T2	Cp22	rho22	mu22
p2	T3	Cp23	rho23	mu23

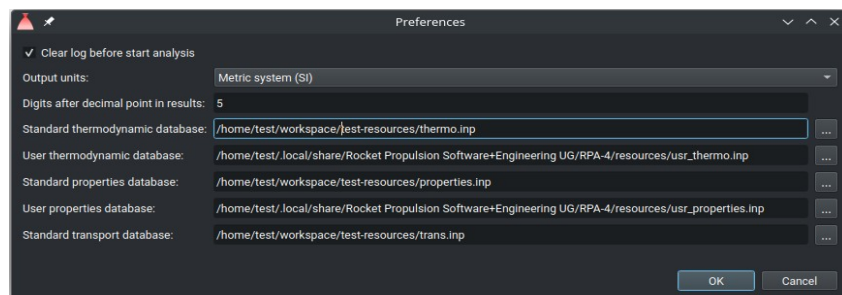
The minimalist data for the component consists of at least two rows:

p, MPa	T, K	Cp, kJ/mol-K	rho, kg/m3	mu, muPa-s
Comp_name	1,2			
0.101325	273.15	0.078	823.0	
0.101325	373.15	0.078	823.0	

This data defines the constant specific heat C_p and constant density ρ , and allows to specify the initial temperature in the range 273.15-373.15 K as well as the initial pressure in the range 0...max-required-pressure. Viscosity is not defined and assumed to be equal 0.

Preferences

Dialog window *Preferences* can be used to set up global configuration parameters. Click the item *Help*, and then *Preferences* in main bar to open the window:



Dialog window "Preferences"

Mark the check box "*Clear log before start analysis*" to clear the analysis log before each analysis run. Leave it unmarked to accumulate the messages from all runs.

The user can select the default output units for results of analysis by choosing either the *Metric system (SI)* or the *U.S. Customary system* from the list Output units.

If selected *Metric system (SI)*, the following units will be used:

Parameter	Unit
Temperature	K (of reaction products)
Temperature	K (of propellant components)
Pressure	MPa
Specific impulse	m/s

Rocket Propulsion Analysis v.4.0.0

Parameter	Unit
Velocity	m/s
Mass flow rate	kg/s
Mass flux	kg/m ² ·s
Thrust	N, kN
Density	kg/m ³
Enthalpy	kJ/kg, J/mol
Entropy, Specific heat, Gas constant	kJ/kg·K, J/mol·K

If selected *U.S. Customary system*, the following units will be used:

Parameter	Unit
Temperature	F (of reaction products)
Temperature	R (of propellant components)
Pressure	psi
Specific impulse	ft/s
Velocity	ft/s
Mass flow rate	lbm/s
Mass flux	lbm/ft ² ·s
Thrust	lbf
Density	lbm/ft ³
Enthalpy	Btu/lbm, Btu/lb·mol
Entropy, Specific heat, Gas constant	Btu/lbm·R, Btu/lb·mol·R

In order to change the number of digits after decimal points in the result tables, update the field *Digits after decimal point in results*.

In order to check for available software updates, click the item **Help** in the menu bar, and then click **Check for Updates**.

Input and Output Units

The user can enter the values of input parameters in any supported units, mixing *Metric (SI)* and *U.S. Customary* systems. The program automatically converts all entered values to the Metric system, which is a standard internal representation of both input parameters and results of calculation.

The following conversion factors are used to convert values in non-SI units to values in SI

Rocket Propulsion Analysis v.4.0.0

units:

Name	Value	Description
CONST_ATM	101325.0	Conversion factor from atm to Pa
CONST_AT	98066.5	Conversion factor from at (technical atmosphere) to Pa
CONST_BAR	100000.0	Conversion factor from bar to Pa
CONST_PSI	6894.75729316836	Conversion factor from psi (pound-force per square inch) to Pa
CONST_T0	298.15 K	Temperature 25 C
CONST_R0	8.314472 J/(mol·K)	Universal Gas Constant
CONST_G	9.80665 m/s ²	
CONST_POUND	0.45359237	Conversion factor from lbm (pound mass) to kg
CONST_POUND_FORCE	(CONST_POUND*CONST_G)	Conversion factor from lbf (pound-force) to N (newton)
CONST_FOOT	0.3048	Conversion factor from international foot to m
CONST_INCH	0.0254	Conversion factor from inch to m
CONST_MILE	(5280.0*CONST_FOOT)	Conversion factor from international mile to m
CONST_LBM_FOOT3	(CONST_POUND/CONST_FOOT ³)	Conversion factor from "lbm/ft ³ " to "kg/m ³ "
CONST_LBM_INCH3	(CONST_POUND/CONST_INCH ³)	Conversion factor from "lbm/inch ³ " to "kg/m ³ "
CONST_MASS_FLUX	(CONST_POUND/CONST_FOOT ²)	Conversion factor from "lbm/(ft ² ·s)" to "kg/(m ² ·s)"
CONST_LBM_MOLE	(1000.*CONST_LBM)	Conversion factor from lb-mole to mole
CONST_BTU	1055.05585262	Conversion factor from Btu to J
CONST_CAL	4.1868	Conversion factor from calorie to J
CONST_BTU_LBM	(CONST_BTU/CONST_LBM)	Conversion factor from Btu/lbm to J/kg
CONST_BTU_LBM_MOLE	(CONST_BTU/CONST_LBM_MOLE)	Conversion factor from Btu/lb-mol to J/mol
CONST_BTU_LBM_R	(1000.*CONST_CAL)	Conversion factor from Btu/(lbm·R) to J/(kg·K)
CONST_BTU_LBM_F	CONST_BTU_LBM_R	Conversion factor from Btu/(lbm·F) to J/(kg·K)
CONST_BTU_LBM_MOLE_R	CONST_BTU_LBM_R	Conversion factor from Btu/(lb-mol·R) to J/(mol·K)

The Metric system is also used by default to display the results of calculation.

References:

- SP-811. NIST Guide for the Use of the International System of Units (SI).
- Glossary of terms and table of conversion factors used in design of chemical propulsion systems. NASA SP-8126. 1979.
- George P. Sutton, Oscar Biblarz. Rocket Propulsion Elements, 7th Edition (pp.727-729).
- NASA-STD-3000. Man-Systems Integration Standards. Volume II. Appendix E - Units of Measure and Conversion Factors.

Scripting API

The scripting utility offers bindings to many internal functions of RPA, allowing users to program and run the following tasks:

- Load, manipulate and write configuration files
- Search the thermodynamic database by species name
- Retrieve thermodynamic properties of species
- Prepare mono- bi- and multi-propellant compositions
- Run typical combustion problems $(p, H) = \text{const}$, $(p, S) = \text{const}$, $(p, T) = \text{const}$
- Run typical rocket propulsion problems
- Utilize all the features listed above in a custom JavaScript program

The scripting utility can be started in either interactive mode or batch mode.

The scripting utility supports JavaScript (ECMAScript) language as defined in standard [ECMA-262](#).

API Reference

Besides standard [ECMA-262 API](#), RPA Scripting Utility provides the domain-specific API.

Generic built-In Functions / Objects

Function / Object	Description
args	JavaScript array containing command-line arguments for the script started in batch mode (using option "--input"). Empty if no arguments specified in the command line.
load(path_to_script)	Load external script defined by it's path. Example: load("resources/scripts/config.js");

Rocket Propulsion Analysis v.4.0.0

Function / Object	Description
<code>print(parameter)</code>	Print out the parameter in console and log file. Examples: <pre>print("Starting analysis..."); print("Is_v=" + pr.getNozzleExitSection().getIs_v().toPrecision(7));</pre>
<code>sprintf(format, parameters)</code>	The function is a limited JavaScript implementation of sprintf, originated from the C programming language. Function returns the string with passed arguments formatted by the usual sprintf conventions. Possible format values: %% – returns a percent sign %b – binary number %c – the character according to the ASCII value %d – signed decimal number %f – floating-point number %o – octal number %s – string %x – hexadecimal number (lowercase letters) %X – hexadecimal number (uppercase letters) Additional format values, placed between the % and the letter (example %.2f): + – forces both + and - in front of numbers. By default, only negative numbers are marked - – left-justifies the variable value 0 – zero will be used for padding the results to the right string size [0-9] – specifies the minimum width held of to the variable value .[0-9] – specifies the number of decimal digits or maximum string length Example: <pre>var s = sprintf("Isp (vac)=%8.4f Isp (SL)=%8.4f s\n", Is_v, Is_SL);</pre>
<code>printf(format, parameters)</code>	The function is a limited Javascript implementation of printf, originated from the C programming language. Function prints out in console and log file the passed arguments formatted by the usual printf conventions. Possible format values: see description of function sprintf. Example: <pre>printf("Isp (vac)=%8.4f Isp (SL)=%8.4f s\n", Is_v, Is_SL);</pre>
<code>sscanf(format, parameters)</code>	The function is a limited Javascript implementation of sscanf, originated from the C programming language. Reads data from "str" and stores them according to parameter "format": a) into array which is returned as a function result (if no additional arguments

Rocket Propulsion Analysis v.4.0.0

Function / Object	Description
	specified), b) into the locations given by the additional arguments (and returns number of scanned parameters). Since JavaScript does not support scalar reference variables, any additional arguments to the function will only be allowable here as strings referring to a global variable (which will then be set to the value found in 'str' corresponding to the appropriate conversion specification in 'format'. Possible format values: %% – A % followed by another % matches a single %. %i – integer %c – the character according to the ASCII value %d, %D – signed decimal number (integer) %u – unsigned decimal number (integer) %f, %e – floating-point number %o – octal number (integer) %s – string %x, %X – hexadecimal number Examples: <pre>sscanf('ls_v=350.0 s', 'ls_v=%f s'); // returns an array with one element: [350.0]</pre> <pre>var ls_v; sscanf('ls_v=350.0 s', 'ls_v=%f s', 'ls_v'); // returns number of scanned values: 1 // assigns the scanned value 350.0 to variable ls_v</pre> <pre>sscanf("ls_v=350.0 s ls_SL=290.0 s", "ls_v=%f s ls_SL=%f s"); // returns an array with two elements: [350.0, 290.0]</pre>
exit() quit() exit quit	Exits from the interactive interpreter.

Object File

Function	Description
File(<i>path_to_file</i> , <i>mode</i>)	Open the file specified by name in given mode. Supported modes: "w" – write file, truncate the content if file exists "wa" – write file, append to existing content if file exists "r" – read file Example: <pre>var f = File("C:\tmp\results.txt", "w");</pre>
print(<i>parameter</i>)	Print out the parameter into the file.

Rocket Propulsion Analysis v.4.0.0

Function	Description
	Examples: <pre>var f = File("C:\tmp\results.txt", "w"); f.print("Starting analysis..."); f.print("Is_v=" + pr.getNozzleExitSection().getIs_v().toPrecision(7)); f.close();</pre>
<code>printf(format, parameters)</code>	The function is a limited Javascript implementation of printf, originated from the C programming language. Function prints out into the file the passed arguments formatted by the usual printf conventions. Possible format values: see description of function sprintf. Example: <pre>var f = File("C:\tmp\results.txt", "w"); f.printf("Isp (vac)=%8.4f Isp (SL)=%8.4f s\n", Is_v, Is_SL); f.close();</pre>
<code>readLine()</code>	The function reads the next available line from the file and returns it as a string. The function returns null if no string is available. Example: <pre>var f = File("C:\tmp\results.txt", "r"); var s = f.readLine(); f.close();</pre>
<code>close()</code>	The function closes the file. Example: <pre>var f = File("C:\tmp\results.txt", "w"); f.printf("Isp (vac)=%8.4f Isp (SL)=%8.4f s\n", Is_v, Is_SL); f.close();</pre>

Object ConfigFile

This object represents the configuration file and provides the functions to access different sub-configurations.

Function	Parameters	Description
<code>ConfigFile()</code>	-	Default constructor. Creates new empty configuration object. Example:

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
		<code>c = ConfigFile();</code>
<code>ConfigFile(String path)</code>	File path	Constructor. Creates new empty configuration object and assign the specified file. The assigned file can be loaded with function <code>read()</code>. Example: <pre>c = ConfigFile("examples/test-1.cfg");</pre>
<code>read(String path)</code>	File path	Reads the specified file. Example: <pre>c = ConfigFile(); c.read("examples/test-1.cfg");</pre>
<code>write(String path)</code>	File path	Writes the configuration into specified file. Example: <pre>c = ConfigFile(); c.read("examples/test-1.cfg"); c.write("examples/test-2.cfg");</pre>
Number <code>getVersion()</code>	-	Returns the version of configuration file. Example: <pre>c = ConfigFile("examples/test-1.cfg"); print("Version: "+c.getVersion());</pre>
<code>getPath()</code>	File path	Assigns the engine name. Example: <pre>c = ConfigFile("examples/test-1.cfg"); print("Path: "+c.getPath());</pre>
String <code>toString()</code>	-	Converts the current configuration to string containing JSON representation of the configuration.

Function	Parameters	Description
		Example: <pre>c = ConfigFile("examples/test-1.cfg"); raw = JSON.parse(cfg.toString()); raw.engineDefinition.chamberPressure .value = 10; raw.engineDefinition.chamberPressure .units = "MPa"; cfg.fromString(JSON.stringify(raw)); print("Problem name: "+c.getName());</pre>
String fromString(String)	JSON string representing configuration	Loads the content from the provided string containing JSON representation of the configuration. Example: <pre>c = ConfigFile("examples/test-1.cfg"); raw = JSON.parse(cfg.toString()); raw.engineDefinition.chamberPressure .value = 10; raw.engineDefinition.chamberPressure .units = "MPa"; cfg.fromString(JSON.stringify(raw)); print("Problem name: "+c.getName());</pre>

Object Units

The object has no constructor, and cannot be instantiated explicitly. Instead, the Scripting Utility provides the global variable “Units”.

Function	Parameters	Description
double convertPressure(double fromValue, String fromUnit, String toUnit)	Pressure value pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Converts pressure value from one value to another. Example: <pre>print(Units.convertPressure(1, "atm", "MPa"));</pre>
double convertTemperature(double fromValue, String fromUnit, String toUnit)	Temperature value temperature unit (one of "K", "F", "C") temperature unit	Converts temperature value from one value to another. Example:

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	(one of "K", "F", "C")	print(Units.convertTemperature(1000, "C", "K"));
double convertMassFlux(double fromValue, String fromUnit, String toUnit)	Mass flux value Mass flux unit (one of "kg/(m2·s)", "kg/(m2-s)", "lbm/(ft2·s)", "lbm/(ft2-s)") Mass flux unit (one of "kg/(m2·s)", "kg/(m2-s)", "lbm/(ft2·s)", "lbm/(ft2-s)")	Converts mass flux value from one value to another.
double convertMassFlowrate(double fromValue, String fromUnit, String toUnit)	Mass flow rate value Mass flow rate unit (one of "kg/s", "lbm/s") Mass flow rate unit (one of "kg/s", "lbm/s")	Converts mass flow rate value from one value to another.
double convertForce(double fromValue, String fromUnit, String toUnit)	Force value Force unit (one of "kN", "N", "kg", "lbf") Force unit (one of "kN", "N", "kg", "lbf")	Converts force value from one value to another.
double convertLength(double fromValue, String fromUnit, String toUnit)	Length value Length unit (one of "m", "mm", "in", "ft") Length unit (one of "m", "mm", "in", "ft")	Converts length value from one value to another.
double convertVelocity(double fromValue, String fromUnit, String toUnit)	Velocity value Velocity unit (one of "m/s", "ft/s", "in/s")	Converts velocity value from one value to another.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	Velocity unit (one of "m/s", "ft/s", "in/s")	
double convertRotationalSpeed(double fromValue, String fromUnit, String toUnit)	Rotational speed value Rotational speed unit (one of "rad/s", "rpm", "Hz") Rotational speed unit (one of "rad/s", "rpm", "Hz")	Converts rotational speed value from one value to another.
double convertThermalConductivity(double fromValue, String fromUnit, String toUnit)	Thermal conductivity value Thermal conductivity unit (one of "W/(m·K)", "W/(m-K)", "Btu/(s·in·R)", "Btu/(s-in-R)", "Btu/(hr·ft·R)", "Btu/(hr-ft-R)") Thermal conductivity unit (one of "W/(m·K)", "W/(m-K)", "Btu/(s·in·R)", "Btu/(s-in-R)", "Btu/(hr·ft·R)", "Btu/(hr-ft-R)")	Converts thermal conductivity value from one value to another.
double convertAngle(double fromValue, String fromUnit, String toUnit)	Angle value Angle unit (one of "rad", "deg") Angle unit (one of "rad", "deg")	Converts angle value from one value to another.
double convertVolumetricMassDensity(double fromValue, String fromUnit, String toUnit)	Volumetric mass density value Volumetric mass density unit (one of "kg/m^3", "g/cm^3", "lbm/in^3",	Converts volumetric mass density value from one value to another.

Function	Parameters	Description
	"lbm/ft^3") Angle unit (one of "kg/m^3", "g/cm^3", "lbm/in^3", "lbm/ft^3")	

Object Database

The object represents the species database. The object has no constructor, and cannot be instantiated explicitly. Instead, the Scripting Utility provides the global variable "Database".

Function	Parameters	Description
Object species(String name)	Name of species	Returns <i>Species</i> object. Example: print(Database.species("H2O2"));

Object Species

The object represents the species. The object has no constructor, and cannot be instantiated explicitly. The instance of the object can be obtained from objects *Database* or *Mixture*.

Function	Parameters	Description
String getName()	-	Returns species name.
String getDescr()	-	Returns species description.
Boolean isReactantOnly()	-	Returns true if species could not be usually used as a propellant component.
Boolean isIon()	-	Returns true if species is ionized.
Number getCharge()	-	Returns the charge of ionized species.
Number getValence()	-	Returns the valency of species.
Boolean isCondensed()	-	Returns true if species is condensed.
Number getCondensed()	-	Returns the number of condensed phase in increased order by temperature.
Number getDHf298_15(String unit)	heat of formation unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns $H^0(298.15)$ - heat of formation at the temperature 298.15 K and pressure 1 bar in desired unit.
Number getDH298_15_0(String unit)	heat of formation unit (one of "J/mol", "Btu/lb-	Returns $H^0(298.15) - H^0(0)$, if available.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	mol", "J/kg", "kJ/kg", "Btu/lbm")	
Number getT0(String unit)	temperature unit (one of "K", "F", "C")	Returns standard temperature of the species in desired unit.
Number getP0(String unit)	pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns standard pressure of the species in desired unit.
Number getMinimumT(String unit)	temperature unit (one of "K", "F", "C")	Returns minimum temperature of the species in desired unit.
Number getMaximumT(String unit)	temperature unit (one of "K", "F", "C")	Returns maximum temperature of the species in desired unit.
Number checkT(double t, String unit)	t - temperature unit - temperature unit (one of "K", "F", "C")	Checks whether the specified temperature is valid. If valid, returns specified temperature. Otherwise throws an exception.
Number getM()		Returns molecular weight of species.
Number getR(String unit)	gas constant unit (one of "J/(mol K)", "J/(kg K)", "kJ/(kg K)", "Btu/(lb-mol R)", "Btu/(lbm R)")	Returns gas constant in desired unit (applicable for gaseous species).
Number getCp(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity at specified temperature and constant pressure in desired unit.
Number getH(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - enthalpy	Returns specific or molar enthalpy at specified temperature in desired unit.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	
Number getS(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)")	Returns specific or molar entropy at specified temperature in desired unit.
Number getG(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - Gibbs energy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns Gibbs energy at specified temperature in desired unit.

Object Mixture

The object represents the mixture ingredients or combustion products. The object can be instantiated using constructor or obtained from other objects.

Function	Parameters	Description
Mixture()	-	Constructor. Creates an empty mixture.
Object addSpecies(String name, Number r)	name - species name r - mass fraction of the species in component	Adds species into mixture, assigns specified mass fraction, and returns Species object.
addMixture(Object mixture, Number r)	Mixture - Mixture object r - mass fraction of the mixture in component	Adds mixture into this mixture object and assigns specified mass fraction.
Number getValence()	-	Returns the resulting valency of mixture.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
Number <code>getEquivalenceRatio()</code>	-	Returns the equivalence ratio of mixture (for more info see Attachments).
Number <code>getEquivalenceRatio(Array omit)</code>	Java script array of chemical elements to be omitted from calculation of equivalent ratio.	Returns the equivalence ratio of mixture (for more info see Attachments). Example: <pre>omit = ["CU"]; printf("Equivalence ratio: %8.6f (omitted: %s)\n", mix.getEquivalenceRatio(omit), omit);</pre>
Number <code>getOxygenBalance()</code>	-	Returns the oxygen balance of mixture (for more info see Attachments).
Number <code>getOxygenBalance(Array omit)</code>	Java script array of chemical elements to be omitted from calculation of oxygen balance.	Returns the oxygen balance of mixture (for more info see Attachments). Example: <pre>omit = ["CU"]; printf("Oxygen balance: %8.6f (omitted: %s)\n", mix.getOxygenBalance(omit), omit);</pre>
Array <code>getFormula(String base)</code>	"g" if number of elements in exploded formula have to be based on 100 g of mixture. Any other string except "g", if number of elements in exploded formula have to be based on 1 mole of mixture.	Returns exploded chemical formula of the mixture as an array of elements. Each element of the array represents one chemical element (atom) with properties "name" and "n".
Number <code>getM()</code>	-	Returns molecular weight of mixture.
Number <code>getH(String unit)</code>	enthalpy unit (one of "J/mol" (default), "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar enthalpy of mixture in desired unit.
Number <code>getU(String unit)</code>	enthalpy unit (one of "J/mol" (default), "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar internal energy of mixture in desired unit.
Number <code>getRho(String unit)</code>	one of "kg/m ³ " (default),	Returns volumetric density of mixture in desired unit.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	"g/cm ³ ", "lbm/ft ³ ", "lbm/in ³ "	
setT(Number t, String unit)	t - temperature unit - temperature unit (one of "K", "F", "C")	Assigns temperature to all species of the mixture.
setP(Number p, String unit)	t - temperature punit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Assigns temperature to all species of the mixture.
Number size()	-	Returns number of species included into mixture.
Object getSpecies(Number i)	index	Returns <i>Species</i> object.
Boolean checkFractions()	-	Returns true if mass fractions assigned correctly (i.e. the sum of all mass fraction within each component is equals to 1.0).
Number getFraction(Number i)	index	Returns assigned mass fraction.
setFraction(Number i, Number f)	i - index f - mass fraction	Assigns mass fraction.
print(String units)	desired units (one of "SI" or "US")	Prints out the information about mixture in desired units.

Object Propellant

The object represents the propellant. The object can be instantiated using constructor or obtained from other objects.

Function	Parameters	Description
Propellant()	-	Constructor. Creates an empty mixture.
Propellant(Number of_ratio)	of_ratio - propellant components mass ratio (O/F)	Constructor. Creates an empty mixture with specified O/F ratio.
setRatio(Number r, String type)	r - propellant components ratio type - type of ratio (one of "O/F", "alpha")	Assigns propellant ratio.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
Object addOxidizer(String name, Number r)	name - species name r - mass fraction of the species in component	Adds species into oxidizer mixture, assigns specified mass fraction, and returns Species object.
Object addFuel(String name, Number r)	name - species name r - mass fraction of the species in component	Adds species into fuel mixture, assigns specified mass fraction, and returns Species object.
Boolean checkFractions(Boolean assign)	Assign - if „true“, the incorrect fractions are fixed to make the sum=1.0	Returns true if mass fractions of both oxidizer and fuel mixtures are assigned correctly (i.e. the sum of all mass fraction within each component is equals to 1.0).
Number getM()	-	Returns molecular weight of propellant.
Number getH(String unit)	enthalpy unit (one of "J/mol" (default), "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar enthalpy of propellant in desired unit.
Number getU(String unit)	enthalpy unit (one of "J/mol" (default), "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar internal energy of propellant in desired unit.
Number getRho(String unit)	one of "kg/m ³ " (default), "g/cm ³ ", "lbm/ft ³ ", "lbm/in ³ "	Returns volumetric density of propellant in desired unit.
Number size(String type)	type - type of component (one of "ox" or "f")	Returns number of species included into component of propellant.
Object getSpecies(String type, Number i)	type - type of component (one of "ox" or "f") i - index	Returns <i>Species</i> object.
Object getOxidizer()	-	Returns <i>Mixture</i> object.
Object getFuel()	-	Returns <i>Mixture</i> object.

Object Equilibrium

Function	Parameters	Description
Equilibrium(Object p, Boolean multiphase, Boolean ionization)	P - <i>Propellant</i> or <i>Mixture</i> object	Constructor.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	multiphase - multiphase flag ionization - ionization flag	Creates Reaction object, using specified Propellant or Mixture object. If multi-phase flag is true, phase transitions effects are considered. If ionization flag is true, ionization effects are considered.
setP(Number p, String p_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") T - temperature T_unit - temperature unit (one of "K", "F", "C")	Assigns pressure of combustion and switches the solving reaction problem to type (p,H)=const. Assigned enthalpy is obtained implicitly from propellant or mixture.
setPT(Number p, String p_unit, Number T, String T_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") T - temperature T_unit - temperature unit (one of "K", "F", "C")	Assigns pressure and temperature of combustion and switches the solving reaction problem to type (p,T)=const.
setPH(Number p, String p_unit, Number H, String H_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") H - enthalpy H_unit - enthalpy unit (one of "J/mol", "Btu/lb-mol", "kJ/mol")	Assigns combustion pressure and enthalpy of propellant and switches the solving reaction problem to type (p,H)=const.
setPS(Number p, String p_unit, Number S, String S_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Assigns combustion pressure and entropy of propellant and switches the solving reaction problem to type (p,S)=const.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	S - entropy S_unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "kJ/(mol K)")	
setV(Number v, String v_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm")	Assigns combustion specific volume (1/rho) and switches the solving reaction problem to type (v,U)=const. Internal energy is obtained implicitly from propellant or mixture.
setVT(Number v, String v_unit, Number T, String T_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm")	Assigns specific volume (1/rho) and temperature of combustion and switches the solving reaction problem to type (v,T)=const.
setVU(Number v, String v_unit, Number U, String U_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm") U - internal energy U_unit - internal energy unit (one of "J/mol", "Btu/lb-mol", "kJ/mol")	Assigns combustion specific volume (1/rho) and internal energy of propellant and switches the solving reaction problem to type (v,U)=const.
setVS(Number v, String v_unit, Number S, String S_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm") S - entropy S_unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "kJ/(mol K)")	Assigns combustion specific volume (1/rho) and entropy of propellant and switches the solving reaction problem to type (v,S)=const.
Object solve(Boolean startWithCondensed)	startWithCondensed flag; if not specified default	Solves the prepared problem. In some cases the problem does not converge because condensed species

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	value is false.	not included before first iteration. To solve such a problems, set <code>startWithCondensed</code> to true. On success, returns the object <i>Derivatives</i> .
<code>reset(Boolean startWithCondensed)</code>	<code>startWithCondensed</code> flag; if not specified default value is false.	Reset the problem before repeating the solving. In some cases the problem does not converge because condensed species not included before first iteration. To solve such a problems, set <code>startWithCondensed</code> to true.
<code>getP(String unit)</code>	pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns assigned pressure.
<code>getV(String unit)</code>	specific volume unit (one of "m ³ /kg", "ft ³ /lbm")	Returns assigned specific volume.
<code>getT(String unit)</code>	temperature unit (one of "K", "F", "C")	Returns the temperature of reaction.
<code>getH(String unit)</code>	enthalpy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar enthalpy of reaction products.
<code>getU(String unit)</code>	internal energy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar internal energy of reaction products.
<code>getS(String unit)</code>	entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)")	Returns specific or molar entropy of reaction products.
Boolean <code>hasCondensedPhase()</code>	-	Returns <i>true</i> if reaction products contains condensed species.
Object <code>getResultingMixture()</code>	-	Returns <i>Mixture</i> object, containing all products of reaction. Note that function <i>Mixture.getSpecies()</i> actually returns <i>Product</i> object.
<code>print(String units)</code>	desired units (one of "SI" or "US")	Prints out the information about problem results.

Object Derivatives

The object represents the thermodynamic derivatives. The object has constructor, but can also be instantiated implicitly from object *Equilibrium*.

Function	Parameters	Description
Derivatives(Object mixture, Number T, Number p)	mixture - Mixture object T - temperature in K p - pressure in Pa	Constructor. Creates new object <i>Derivatives</i> for given <i>Mixture</i> object. If temperature and pressure are not specified, the values assigned to mixture are used.
Number getCp(String unit)	specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity of reaction products at constant pressure in desired unit.
Number getCv(String unit)	specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity of reaction products at constant volume in desired unit.
Number getR(String unit)	gas constant unit (one of "J/(mol K)", "J/(kg K)", "kJ/(kg K)", "Btu/(lb-mol R)", "Btu/(lbm R)")	Returns gas constant of reaction products in desired unit.
Number getK()	-	Returns isentropic exponent of reaction products.
Number getGamma()	-	Returns specific heat ratio of reaction products.
Number getA(String unit)	velocity unit (one of "m/s" or "ft/s")	Returns velocity of sound in desired unit.
Number getRho(String unit)	density unit (one of "kg/m ³ ", "g/m ³ " or "lbm/ft ³ ")	Returns density of reaction products in desired unit.
Number getRhoGas(String unit)	density unit (one of "kg/m ³ ", "g/m ³ " or "lbm/ft ³ ")	Returns density of gaseous reaction products in desired unit.
Number getZ()	-	Returns mass fraction of condensed

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
		reaction products.
Number getM()	-	Returns molecular weight of reaction products.
Number getV(String unit)	unit (one of "x10 ⁻⁴ kg/(m s)", "mpoise" or "x10 ⁻⁴ lb/(ft s)")	Returns viscosity of reacting mixture.
Number getCr(String unit)	unit (one of "W/(m K)", "mW/(cm K)" or "Btu/(hr ft R)")	Returns equilibrium conductivity of reacting mixture.
Number getCfr(String unit)	unit (one of "x10 ⁻⁴ kg/(m s)", "mpoise" or "x10 ⁻⁴ lb/(ft s)")	Returns frozen conductivity of reacting mixture.
Number getCeql(String unit)	unit (one of "x10 ⁻⁴ kg/(m s)", "mpoise" or "x10 ⁻⁴ lb/(ft s)")	Returns effective conductivity of reacting mixture.
Number getCpr(String unit)	unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns equilibrium specific heat of reacting mixture at constant pressure
Number getCpfr(String unit)	unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns frozen specific heat of reacting mixture at constant pressure
Number getCpeql(String unit)	unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns effective specific heat of reacting mixture at constant pressure
Number getPr()	-	Returns equilibrium Prandtl number (<u>Pr</u>) of reacting mixture.
Number getPrfr()	-	Returns frozen Prandtl number (<u>Pr</u>) of reacting mixture.
Number getPreql()	-	Returns effective Prandtl number (<u>Pr</u>) of reacting mixture.
print(String units)	desired units (one of "SI" or "US")	Prints out the information derivative properties.

Object Product

The object represents the reaction product. The object has no constructor, and cannot be instantiated explicitly.

Function	Parameters	Description
String getName()	-	Returns species name.
String getDescr()	-	Returns species description.
getN()	-	Returns the mole quantity of the product in reacting mixture.
Number getT(String unit)	temperature unit (one of "K", "F", "C")	Returns temperature of the product in desired unit.
Boolean isReactantOnly()	-	Returns true if species could not be usually used as a propellant component.
Boolean isIon()	-	Returns true if species is ionized.
Number getCharge()	-	Returns the charge of ionized species.
Number getValence()	-	Returns the valency of species.
Boolean isCondensed()	-	Returns true if species is condensed.
Number getCondensed()	-	Returns the number of condensed phase in increased order by temperature.
Number getDHf298_15(String unit)	heat of formation unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns $H^0(298.15)$ - heat of formation at the temperature 298.15 K and pressure 1 bar in desired unit.
Number getDH298_15_0(String unit)	heat of formation unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns $H^0(298.15) - H^0(0)$, if available.
Number getT0(String unit)	temperature unit (one of "K", "F", "C")	Returns standard temperature of the species in desired unit.
Number getP0(String unit)	pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns standard pressure of the species in desired unit.
Number getMinimalT(String unit)	temperature unit (one of "K", "F", "C")	Returns minimum temperature of the species in desired unit.
Number getMaximalT(String unit)	temperature unit (one of "K", "F", "C")	Returns maximum temperature of the species in desired unit.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	"C")	
Number checkT(double t, String unit)	t - temperature unit - temperature unit (one of "K", "F", "C")	Checks whether the specified temperature is valid. If valid, returns specified temperature. Otherwise throws an exception.
Number getM()	-	Returns molecular weight of species.
Number getR(String unit)	gas constant unit (one of "J/(mol K)", "J/(kg K)", "kJ/(kg K)", "Btu/(lb-mol R)", "Btu/(lbm R)")	Returns gas constant in desired unit (applicable for gaseous species).
Number getCp(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity at specified temperature and constant pressure in desired unit.
Number getH(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - enthalpy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar enthalpy at specified temperature in desired unit.
Number getS(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)")	Returns specific or molar entropy at specified temperature in desired unit.

Function	Parameters	Description
Number getG(Number t, String tunit, String unit)	t - temperatur tunit - temperature unit (one of "K", "F", "C") unit - Gibbs energy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns Gibbs energy at specified temperature in desired unit.

Object Combustor

The object *Combustor* can be used to solve different combustion problems. It does not applicable to rocket propulsion problems involving nozzle flow and rocket performance.

Function	Parameters	Description
Combustor(Object p, Boolean multiphase, Boolean ionization)	p - <i>Propellant</i> or <i>Mixture</i> object multiphase - multiphase flag ionization - ionization flag	Constructor. Creates <i>Combustor</i> object using given object <i>Propellant</i> or <i>Mixture</i>. If multi-phase flag is true, phase transitions effects are considered. If ionization flag is true, ionization effects are considered.
setP(Number p, String unit)	p - pressure pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Assigns combustion chamber pressure. Enthalpy of propellant or mixture is automatically assigned.
setPH(Number p, String p_unit, Number H, String H_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") H - enthalpy H_unit - enthalpy unit (one of "J/mol", "Btu/lb-mol", "kJ/mol")	Assigns combustion chamber pressure and enthalpy of mixture of ingredients.
setPT(Number p, String p_unit, Number T, String T_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Assigns pressure and temperature of combustion and switches the solving reaction problem to type (p,T)=const.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	T - temperature T_unit - temperature unit (one of "K", "F", "C")	
setPS(Number p, String p_unit, Number S, String S_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") S - entropy S_unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "kJ/(mol K)")	Assigns combustion pressure and entropy of propellant and switches the solving reaction problem to type (p,S)=const.
setVT(Number v, String v_unit, Number T, String T_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm")	Assigns specific volume (1/rho) and temperature of combustion and switches the solving reaction problem to type (v,T)=const.
setVU(Number v, String v_unit, Number U, String U_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm") U - internal energy U_unit - internal energy unit (one of "J/mol", "Btu/lb-mol", "kJ/mol")	Assigns combustion specific volume (1/rho) and internal energy of propellant and switches the solving reaction problem to type (v,U)=const.
setVS(Number v, String v_unit, Number S, String S_unit)	v - specific volume v_unit - specific volume unit (one of "m^3/kg", "ft^3/lbm") S - entropy S_unit - entropy unit (one of	Assigns combustion specific volume (1/rho) and entropy of propellant and switches the solving reaction problem to type (v,S)=const.

Function	Parameters	Description
	"J/(mol K)", "Btu/(lb-mol R)", "kJ/(mol K)"	
omit(Array species)	species - array of species to be omitted from calculation	Specifies the array of species which should be included into the combustion composition.
solve(Boolean startWithCondensed, Boolean withTransportProperties)	startWithCondensed - true or false withTransportProperties - true or false	Solves the chemical equilibrium problem. In some cases the problem does not converge because condensed species not included before first iteration. To solve such a problems, set startWithCondensed to true.
Object getEquilibrium()	-	Returns object <i>Reaction</i> for solved chemical equilibrium problem.
Object getDerivatives()	-	Returns object <i>Derivatives</i> for solved chemical equilibrium problem.

Object Chamber

The object *Chamber* can be used to solve rocket propulsion problems including nozzle flow and rocket performance.

Function	Parameters	Description
Chamber(Object p, Number fr_condition, String fr_type, Boolean multiphase, Boolean ionization)	p - <i>Propellant</i> or <i>Mixture</i> object fr_condition - condition for freezing chemical equilibrium fr_type - type of condition (one of "Ar", "pr", or "p") multiphase - multiphase flag ionization - ionization flag	Constructor. Creates <i>Chamber</i> object using given object <i>Propellant</i> or <i>Mixture</i> . If freezing condition is specified, the model of frozen equilibrium in the nozzle is applied starting with specified value. If multi-phase flag is true, phase transitions effects are considered. If ionization flag is true, ionization effects are considered.
setMr(Number mr, String unit)	mr - mass flux value unit - mass flux rate (one of "kg/(m ² ·s)", "kg/(m ² -s)",	Assigns combustion chamber mass flux (Mr=m_dot/Ac).

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	"lbm/(ft ² ·s)", "lbm/(ft ² -s)")	
setAcr()	-	Assigns combustion chamber contraction area ratio (Ac/At).
setP(Number p, String unit)	p - pressure pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Assigns combustion chamber pressure. Enthalpy of propellant or mixture is automatically assigned.
setPH(Number p, String p_unit, Number H, String H_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") H - enthalpy H_unit - enthalpy unit (one of "J/mol", "Btu/lb-mol", "kJ/mol")	Assigns combustion chamber pressure and enthalpy of mixture of ingredients.
setPT(Number p, String p_unit, Number T, String T_unit)	p - pressure p_unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") T - temperature T_unit - temperature unit (one of "K", "F", "C")	Assigns pressure and temperature of combustion and switches the solving reaction problem to type (p,T)=const.
omit(Array species)	species - array of species to be omitted from calculation	Specifies the array of species which should be included into the combustion composition.
assignCorrectionFactors(Number reactionEfficiency, Number divergenceEfficiency, Number frictionEfficiency, Number multiphaseEfficiency)		Assigns given correction factors. If not assigned, the code will estimate correction factors using default embedded algorithms.
solve(Boolean startWithCondensed, Boolean withTransportProperties)	startWithCondensed - true or false withTransportProperties - true or	Solves the chemical equilibrium problem. In some cases the problem does not converge because condensed species

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	false	not included before first iteration. To solve such a problems, set startWithCondensed to true.
Number getAcr()	-	Returns the combustion chamber contraction area ratio (A_c/A_t).
Number getMr(String unit)	unit - mass flux rate (one of "kg/(m ² ·s)", "kg/(m ² -s)", "lbm/(ft ² ·s)", "lbm/(ft ² -s)")	Returns the combustion chamber mass flux ($M_r = \dot{m}/A_c$).
Number getPc_0(String unit)	unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns stagnation pressure at nozzle inlet.
Number getSigmaC()	-	Returns coefficient of stagnation pressure loss at nozzle inlet (combustion chamber end).
Number getWc(String unit)	unit - velocity unit (one of "m/s" or "ft/s")	Returns the velocity at nozzle inlet (combustion chamber end).
Number getBetta(String unit)	unit - velocity unit (one of "m/s" or "ft/s")	Returns characteristic velocity (the same as getCstar()).
Number getCstar(String unit)	unit - velocity unit (one of "m/s" or "ft/s")	Returns characteristic velocity (the same as getBetta()).
Object getEquilibrium(String station)	station - one of "inj" or "injector", "inl" or "inlet", "thr" or "throat"	Returns object <i>Equilibrium</i> for specified chamber station.
Object getDerivatives(String station)	station - one of "inj" or "injector", "inl" or "inlet", "thr" or "throat"	Returns object <i>Derivatives</i> for specified chamber station.
Boolean isFrozenConditionSet()	-	Returns „true“, if frozen condition is assigned.
Object getEquilibriumStation()	-	Returns object <i>ChamberStationConditions</i> which corresponds to the last station with equilibrium flow.
Object getNozzleStation(Number condition, String type, Boolean superSonic, Boolean startWithCondensed, Boolean	condition - condition value (depends on type)	Returns object <i>ChamberStationConditions</i> for given conditions.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
withTransportProperties)	type - condition type (one of "Ar", "pr", or "p") superSonic - „true“ if "Ar" (area ratio) condition correspond to station downstream of nozzle throat. Applicable only for condition type "Ar" startWithCondensed - if „true“, the equilibrium will consider condensed species in reaction products in the first iteration; this can be useful for propellants/mixtures containing metals withTransportProperties - if „true“, the results will include transport properties of reaction products	

Object ChamberStationConditions

Function	Parameters	Description
Number getAr()	-	Returns the area ratio (A/A_t) at nozzle station.
Number getPr()	-	Returns the pressure ratio (p/p_c) at nozzle station.
Number getP(String unit)	unit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns the pressure at nozzle station.
Number getW(String unit)	unit - velocity unit (one of "m/s" or "ft/s")	Returns the flow velocity at nozzle station.
Number getMach()	-	Returns the Mach number at nozzle station.
Number getMr(String unit)	unit - mass flux rate (one of	Returns the mass flux ($\dot{m}_r = \dot{m}/A_c$) at nozzle station.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	<code>"kg/(m2·s)",</code> <code>"kg/(m2-s)",</code> <code>"lbm/(ft2·s)",</code> <code>"lbm/(ft2-s)"</code>	
Number getIs_v(String unit, bool applyCorrections)	unit – specific impulse unit (one of "s", "m/s" or "ft/s") applyCorrections – if "true", returns delivered specific impulse	Returns specific impulse in vacuum as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.
Number getIs(String unit, bool applyCorrections)	unit – specific impulse unit (one of "s", "m/s" or "ft/s") applyCorrections – if "true", returns delivered specific impulse	Returns specific impulse at optimum expansion as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.
Number getIs_H(Number pH, String phunit, String unit, bool applyCorrections)	pH – ambient pressure value phunit – ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") unit – specific impulse unit (one of "s", "m/s" or "ft/s") applyCorrections – if "true", returns delivered specific impulse	Returns specific impulse at given ambient pressure as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.
Number getCf_v(bool applyCorrections)	applyCorrections – if "true", returns delivered specific impulse	Returns thrust coefficient in vacuum as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.
Number getCf(bool applyCorrections)	applyCorrections – if "true", returns delivered specific impulse	Returns thrust coefficient at optimum expansion as if the nozzle station is a nozzle exit station.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
		If <code>applyCorrections="false"</code> , returns ideal value; otherwise returns estimated delivered value.
Number <code>getCf_H</code> (Number <code>pH</code> , String <code>phunit</code> , bool <code>applyCorrections</code>)	<code>pH</code> - ambient pressure value <code>phunit</code> - ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") <code>applyCorrections</code> - if "true", returns delivered specific impulse	Returns thrust coefficient at given ambient pressure as if the nozzle station is a nozzle exit station. If <code>applyCorrections="false"</code>, returns ideal value; otherwise returns estimated delivered value.
Object <code>getResultingMixture</code> ()	-	Returns object <i>Mixture</i> containing objects <i>Product</i> representing the products of reaction.
<code>printResults</code> ()	-	Prints the results in console.
Object <code>getEquilibrium</code> ()	-	Returns object <i>Equilibrium</i> for nozzle station, or „undefined“ if nozzle station is located downstream of equilibrium freezing station. Applicable only for station with shifting equilibrium.
Object <code>getDerivatives</code> ()	-	Returns object <i>Derivatives</i> for nozzle station, or „undefined“ if nozzle station is located downstream of equilibrium freezing station.
Number <code>getT</code> (String <code>unit</code>)	temperature unit (one of "K", "F", "C")	Returns the temperature of reaction products.. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions <code>getEquilibrium</code>() and <code>getDerivatives</code>()..
Number <code>getH</code> (String <code>unit</code>)	enthalpy unit (one of "J/mol", "Btu/lb-mol", "J/kg", "kJ/kg", "Btu/lbm")	Returns specific or molar enthalpy of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions <code>getEquilibrium</code>() and <code>getDerivatives</code>()..
Number <code>getS</code> (String <code>unit</code>)	unit - entropy unit (one of "J/(mol K)", "Btu/(lb-mol R)", "J/(kg K)",	Returns specific or molar entropy of reaction products. Applicable only for station with frozen

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	"kJ/(kg K)", "Btu/(lbm R)"	equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getCp(String unit)	unit - specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getCv(String unit)	specific heat unit (one of "J/(kg K)", "kJ/(kg K)", "Btu/(lbm R)") or molar heat capacity unit (one of "J/(mol K)", "Btu/(lb-mol R)")	Returns specific heat or molar heat capacity of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getR(String unit)	gas constant unit (one of "J/(mol K)", "J/(kg K)", "kJ/(kg K)", "Btu/(lb-mol R)", "Btu/(lbm R)")	Returns gas constant of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getK()	-	Returns isentropic exponent of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getA(String unit)	velocity unit (one of "m/s" or "ft/s")	Returns velocity of sound. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getRho(String unit)	density unit (one of "kg/m^3", "g/m^3" or "lbm/ft^3")	Returns density of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium stations, see functions getEquilibrium() and getDerivatives().
Number getM()	-	Returns molecular weight of reaction products. Applicable only for station with frozen equilibrium. For shifting equilibrium

Function	Parameters	Description
		stations, see functions <code>getEquilibrium()</code> and <code>getDerivatives()</code> .

Object TheoreticalPerformance

The object is used to run the performance analysis.

Function	Parameters	Description
<code>TheoreticalPerformance(Object p, Object c)</code>	p – JavaScript object with assigned properties representing <i>PropellantConfiguration</i> c – JavaScript object with assigned properties representing <i>ChamberConfiguration</i>	Constructor the object <i>TheoreticalPerformance</i> .
<code>assignCorrectionFactors(Number reactionEfficiency, Number divergenceEfficiency, Number frictionEfficiency, Number multiphaseEfficiency)</code>		Assigns given correction factors. If not assigned, the code will estimate correction factors using default embedded algorithms.
<code>solve(Boolean withTransportProperties)</code>	withTransportProperties – if „true“, calculates transport properties for each nozzle station	Solves the theoretical rocket performance for given configuration.
<code>Boolean isSolved()</code>	-	Returns „true“, if performance problem is already solved.
<code>Boolean isOptimized()</code>	-	Returns „true“, if propellant component ratio (O/F) was optimized.
<code>Object getChamber()</code>	-	If performance problem was solved successfully, returns the reference to the object <i>Chamber</i> .
<code>Object getNozzleExit()</code>	-	If performance problem was solved successfully, returns the reference to the object <i>ChamberStationConditions</i> for nozzle exit.
<code>Object getNozzleStation(Number condition, String type, Boolean supersonic, Boolean checkForFreezing, Boolean withTransportProperties)</code>	condition – condition value (depends on type) type – condition type (one of "Ar",	Returns object <i>ChamberStationConditions</i> for given conditions.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	"pr", or "p") supersonic - „true“ if "Ar" (area ratio) condition correspond to station downstream of nozzle throat. Applicable only for condition type "Ar" checkForFreezing - if „true“, check whether equilibrium is frozen at the given conditions withTransportProperties - if „true“, the results will include transport properties of reaction products	
Number getPaCrit(String punit)	punit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns ambient pressure where flow separation occurs
Object getOverExpansionStation()	-	Returns nozzle station where flow separation occurs for default ambient pressure specified in ChamberSpecification
Object getOverExpansionStation(Number pa, String punit)	pa - ambient pressure punit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns nozzle station where flow separation occurs for given ambient pressure.
Number getIs_H(Number pa, String punit, Boolean applyCorrections)	pa - ambient pressure value punit - ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") unit - specific impulse unit (one of "s", "m/s" or "ft/s")	Returns specific impulse with respect to flow separation at given ambient pressure as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.

Function	Parameters	Description
	applyCorrections - if "true", returns delivered specific impulse	
Number getCf_H(Number pa, String punit, bool applyCorrections)	pa - ambient pressure value punit - ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") applyCorrections - if "true", returns delivered specific impulse	Returns thrust coefficient with respect to flow separation at given ambient pressure as if the nozzle station is a nozzle exit station. If applyCorrections="false", returns ideal value; otherwise returns estimated delivered value.

Object TheoreticalThrottlingPerformance

The object is used to run the performance analysis for throttled thrust level. All functions of object *TheoreticalPerformance* are inherited and can be used from this object as well. The returned results correspond to the specified throttle level.

Function	Parameters	Description
TheoreticalThrottlingPerformance(Object nominal, Number r)	nominal - reference to object <i>TheoreticalPerformance</i> r - throttling level	Constructor the object <i>TheoreticalThrottlingPerformance</i> for given nominal thrust and throttling level.
Number getR()	-	Returns throttling level
Object getNominalThrustPerformance()	-	Returns object <i>TheoreticalPerformance</i> for nominal thrust.
assignCorrectionFactors(Number reactionEfficiency, Number divergenceEfficiency, Number frictionEfficiency, Number multiphaseEfficiency)		Assignes given correction factors. If not assigned, the code will estimate correction factors using embedded algorithms.
solve(Boolean withTransportProperties)	withTransportProperties - if „true“, calculates transport properties for each nozzle station	Solves the theoretical rocket performance for given configuration.
Boolean isSolved()	-	Returns „true“, if performance problem

Rocket Propulsion Analysis v.4.0.0

<code>assignCorrectionFactors(Number reactionEfficiency, Number divergenceEfficiency, Number frictionEfficiency, Number multiphaseEfficiency)</code>		Assignes given correction factors. If not assigned, the code will estimate correction factors using embedded algorithms.
		is already solved.
<code>Boolean isOptimized()</code>	-	Returns „true“, if propellant component ratio (O/F) was optimized.
<code>Object getChamber()</code>	-	If performance problem was solved successfully, returns the reference to the object <i>Chamber</i> .
<code>Object getNozzleExit()</code>	-	If performance problem was solved successfully, returns the reference to the object <i>ChamberStationConditions</i> for nozzle exit.
<code>Object getNozzleStation(Number condition, String type, Boolean supersonic, Boolean checkForFreezing, Boolean withTransportProperties)</code>	condition - condition value (depends on type) type - condition type (one of "Ar", "pr", or "p") supersonic - „true“ if "Ar" (area ratio) condition correspond to station downstream of nozzle throat. Applicable only for condition type "Ar" checkForFreezing - if „true“, check whether equilibrium is frozen at the given conditions withTransportProperties - if „true“, the results will include transport	Object getNozzleStation(Number condition, String type, Boolean supersonic, Boolean checkForFreezing, Boolean withTransportProperties)
<code>Number getPaCrit(String punit)</code>	punit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	Returns ambient pressure where flow separation occurs
<code>Object getOverExpansionStation()</code>	-	Returns nozzle station where flow separation occurs for default ambient pressure specified in <i>ChamberSpecification</i>
<code>Object</code>	pa - ambient	Returns nozzle station where flow

Rocket Propulsion Analysis v.4.0.0

<code>assignCorrectionFactors(Number reactionEfficiency, Number divergenceEfficiency, Number frictionEfficiency, Number multiphaseEfficiency)</code>		Assignes given correction factors. If not assigned, the code will estimate correction factors using embedded algorithms.
<code>getOverExpansionStation(Number pa, String punit)</code>	pressure punit - pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa")	separation occurs for given ambient pressure.
<code>Number getIs_H(Number pa, String punit, Boolean applyCorrections)</code>	pa - ambient pressure value punit - ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") unit - specific impulse unit (one of "s", "m/s" or "ft/s") applyCorrections - if "true", returns delivered specific impulse	Returns specific impulse with respect to flow separation at given ambient pressure as if the nozzle station is a nozzle exit station. If <code>applyCorrections="false"</code>, returns ideal value; otherwise returns estimated delivered value.
<code>Number getCf_H(Number pa, String punit, bool applyCorrections)</code>	pa - ambient pressure value punit - ambient pressure unit (one of "Pa", "psi", "atm", "bar", "at", "MPa") applyCorrections - if "true", returns delivered specific impulse	Returns thrust coefficient with respect to flow separation at given ambient pressure as if the nozzle station is a nozzle exit station. If <code>applyCorrections="false"</code>, returns ideal value; otherwise returns estimated delivered value.

Object *ChamberDesign*

Object *ChamberDesign* provides functions to specify and calculate combustion chamber geometry. The instance of the object can be obtained from *PropulsionAnalysis*, or constructed directly.

Function	Parameters	Description
<code>ChamberDesign(Object p, Boolean applyCorrections)</code>	p - reference to JavaScript object <i>ChamberDesignConfig</i>	Constructor the object <i>ChamberDesign</i> for given

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	<code>uration</code> with configuration parameters <code>applyCorrections</code> - if "true", apply corrections factors	configuration parameters.
Object <code>getConfiguration()</code>	-	Returns the object <i>ChamberDesignConfiguration</i> with configuration parameters used to construct the object <i>ChamberDesign</i> .
Object <code>getSize(Number contourPoints)</code>	<code>contourPoints</code> - number of contour points	For already calculated design, returns the reference to JavaScript object <i>ChamberDesignResults</i> . Actual number of points can differ from specified one, and is returned in the object <i>ChamberDesignResults</i> .

Object *NozzleDesign*

Function	Parameters	Description
<code>NozzleDesign(Object p)</code>	<code>p</code> - reference to JavaScript object <i>NozzleDesignConfiguration</i> with configuration parameters	Constructor the object <i>NozzleDesign</i> for given configuration parameters.
Object <code>getConfiguration()</code>	-	Returns the object <i>NozzleDesignConfiguration</i> with configuration parameters used to construct the object <i>NozzleDesign</i> .
Object <code>getSize(Boolean startFromLc, Number contourPoints)</code>	<code>startFromLc</code> - if "false", the location of the first contour point is 0. Otherwise, the location of the first point is "Lc" (combustion chamber length). <code>contourPoints</code> - number of contour points	For already calculated design, returns the reference to JavaScript object <i>NozzleDesignResults</i> . Actual number of points can differ from specified one, and is returned in the object <i>NozzleDesignResults</i> .

Object DXF

Object *DXF* has no constructor, and provides static function to generate DXF file for CAD systems from provides objects *ChamberDesign* and *NozzleDesign*.

Function	Parameters	Description
draw(String fileName, String descr, Object chamberDesign, Object nozzleDesign)	fileName - target name to save generated DXF file. descr - description of DXF file. chamberDesign - reference to object <i>ChamberDesign</i>. nozzleDesign - reference to object <i>NozzleDesign</i>.	Generates DXF file.

Object HeatTransferAnalysis

Function	Parameters	Description
HeatTransferAnalysis(Object p)	p - reference to JavaScript object <i>HeatTransferConfiguration</i> with configuration parameters	Constructor the object <i>HeatTransferAnalysis</i> for given configuration parameters.
Object getConfiguration()	-	Returns the object <i>HeatTransferConfiguration</i> with configuration parameters used to construct the object <i>HeatTransferAnalysis</i> .
Object getResults()	-	Returns JavaScript <i>HeatTransferAnalysisResults</i> object with results.

Object PropulsionAnalysis

Object *PropulsionAnalysis* is used for top-level configuration and analysis.

Function	Parameters	Description
PropulsionAnalysis(Object cfg, Boolean printResults, Boolean outputMetric)	cfg - reference to object <i>ConfigFile</i> printResults - if	Constructor the object <i>PropulsionAnalysis</i> for given configuration files.

Rocket Propulsion Analysis v.4.0.0

Function	Parameters	Description
	„true“, the results are printed out in console after completing the analysis outputMetric - if „true“, the results are printed out in SI (metric) units	The analysis is started automatically.
Object getPerformanceAnalysis()	-	Returns object <i>TheoreticalPerformance</i>
Boolean hasAltitudePerformanceAnalysis()	-	Returns „true“, if altitude performance analysis is completed.
Object getAltitudePerformanceAnalysis()	-	Returns the array of JavaScript objects <i>AltitudePerformance</i> .
Boolean hasThrottlingPerformanceAnalysis()	-	Returns „true“ if throttling performance analysis is completed.
Object getThrottlingPerformanceAnalysis()	-	Returns the array of objects <i>TheoreticalThrottlingPerformance</i> .
Number getStationsListSize()	-	
Object getStation(Number n)		
Object getChamberDesign()	-	
Object getChamberSize(Number n)		
Object getNozzleDesign()	-	
Object getNozzleSize(Boolean startFromLc, Number n)		
Object getNozzleHeatTransferAnalysis()	-	

Object AltitudePerformance

Object *AltitudePerformance* is a pure JavaScript object with properties and without functions. It used to return the results of altitude performance analysis.

Property	Description
pa	Ambient pressure, Pa
h	Altitude, m
Is	Ideal specific impulse, m/s
Is_overexp	Ideal specific impulse for overexpanded flow, m/s
Cf	Ideal thrust coefficient

Property	Description
Cf_overexp	Ideal thrust coefficient for overexpanded flow
Is_d	Delivered specific impulse, m/s
Is_overexp_d	Delivered specific impulse for overexpanded flow, m/s
Cf_d	Delivered thrust coefficient
Cf_overexp_d	Delivered thrust coefficient for overexpanded flow
delivered	"true" if parameters for delivered performance are provided

Object *PropellantConfiguration*

Object *PropellantConfiguration* is a pure JavaScript object with properties and without functions. It is used to prepare the configuration parameters to be passed to constructor of objects *TheoreticalPerformance* or *TheoreticalThrottlingPerformance*.

Property	Description
propellant	Reference to object <i>Propellant</i> .
mixture	Reference to object <i>Mixture</i> . Note the property <i>propellant</i> has priority: if both properties <i>propellant</i> and <i>mixture</i> are specified, the property <i>mixture</i> will be ignored.
H	Assigned enthalpy of propellant or mixture. Optional property: if missing, the enthalpy is obtained from assigned propellant or mixture.
H_unit	Unit for assigned enthalpy. One of "kJ/mol", "J/mol", or "Btu/lb-mol"
T	Assigned temperature of propellant or mixture. Optional property: if missing, the standard temperature is assumed.
T_unit	Unit for assigned temperature. One of "K", "F", "C",
optimize	If „true“, the component ratio of propellant (O/F) will be optimized for max specific impulse. Applicable only if property <i>propellant</i> is assigned.
blc_m	Relative mass flow rate ($m_{blc} = m_{dot_blc}/m_{dot_total}$) of Boundary Layer Coolant (BLC).
blc_propellant	Reference to object <i>Propellant</i> for BLC.
blc_H	Assigned enthalpy of propellant or mixture for BLC. Optional property: if missing, the enthalpy is obtained from assigned propellant or mixture.
blc_H_unit	Unit for assigned enthalpy. One of "kJ/mol", "J/mol", or "Btu/lb-mol"
blc_T	Assigned temperature of propellant or mixture for BLC. Optional property: if missing, the standard temperature is assumed.
blc_T_unit	Unit for assigned temperature. One of "K", "F", "C",

Examples:

```
pc = new Object();
pc.propellant = new Propellant(2.6);
pc.propellant.addOxidizer("O2(L)", 1.0);
pc.propellant.addFuel("H2(L)", 1.0);
```

```
propellant = new Propellant(2.6);
propellant.addOxidizer("O2(L)", 1.0);
propellant.addFuel("H2(L)", 1.0);
```

Rocket Propulsion Analysis v.4.0.0

```
pc.optimize = false;

cc = new Object();
cc.pc = 1;
cc.pc_unit = "MPa"
cc.ions = true;
cc.multiphase = false;
cc.condensed = false;
cc.solveNozzleFlow = true;
cc.nozzleInlet_Ar = 2.5;
cc.freezing_Ar = 2;
cc.nozzleExit_Ar = 20;

tp = new TheoreticalPerformance(pc, cc);
tp.solve();
tp.printResults();

pc = {
  "propellant":propellant,
  "optimize":false
};

cc = {
  "pc":1,
  "pc_unit": "MPa",
  "ions": true,
  "multiphase": false,
  "condensed": false,
  "solveNozzleFlow": true,
  "nozzleInlet_Ar": 2.5,
  "freezing_Ar": 2,
  "nozzleExit_Ar":20
};

tp = new TheoreticalPerformance(pc, cc);
tp.solve();
tp.printResults();
```

Object ChamberConfiguration

Object *ChamberConfiguration* is a pure JavaScript object with properties and without functions. It is used to prepare the configuration parameters to be passed to constructor of objects *TheoreticalPerformance* or *TheoreticalThrottlingPerformance*.

Property	Description
pc	Combustion chamber pressure
pc_unit	Pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".
Tc	Assigned combustion chamber temperature. Optional property.
Tc_unit	Temperature unit. One of "K", "F", "C",
ions	If „true“, ionization effects will be considered.
multiphase	If „true“, multi-phase effects will be considered.
condensed	If „true“, condensed phase will be considered during first iteration. Can be useful for propellant/mixture containing metals.
solveNozzleFlow	If „false“, solve combustion problem only without nozzle flow. If „true“, solve the nozzle flow: nozzle inlet, nozzle throat, and nozzle exit stations.
nozzleInlet_Ar	If provided, solve the nozzle inlet with specified contraction area ratio ($Ar=A_c/A_t$). If neither <i>nozzleInlet_Ar</i> nor <i>nozzleInlet_Mr</i> is provided, solve the nozzle inlet flow for infinity-area combustion chamber.

Rocket Propulsion Analysis v.4.0.0

Property	Description
nozzleInlet_Mr	If provided, solve the nozzle inlet with specified mass flux ($M_r = \dot{m}/A_c$). If neither <i>nozzleInlet_Ar</i> nor <i>nozzleInlet_Mr</i> is provided, solve the nozzle inlet flow for infinity-area combustion chamber.
nozzleInlet_Mr_unit	Mass flux unit. One of "kg/(m ² ·s)", "kg/(m2-s)", "lbm/(ft ² ·s)", "lbm/(ft2-s)"
freezing_p	If provided, the model of equilibrium freezing at specified pressure is applied to nozzle flow.
freezing_p_unit	Pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".
freezing_Ar	If provided, the model of equilibrium freezing at specified area ratio ($A_r = A_{fr}/A_t$) is applied to nozzle flow.
freezing_pr	If provided, the model of equilibrium freezing at specified pressure ratio ($p_r = p_c/A$) is applied to nozzle flow.
nozzleExit_p	If provided, nozzle exit is defined by nozzle exit pressure.
nozzleExit_p_unit	Pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".
nozzleExit_Ar	If provided, nozzle exit is defined by nozzle exit area ratio ($A_r = A_e/A_t$).
nozzleExit_pr	If provided, nozzle exit is defined by nozzle exit pressure ratio ($p_r = p_c/p_e$).
solveOverexpansion	If „true“, estimate performance loss due to flow separation in overexpanded flow.
pa	Default ambient pressure used to estimate flow separation.
pa_unit	Ambient pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".

Examples:

```

pc = new Object();
pc.propellant = new Propellant(2.6);
pc.propellant.addOxidizer("O2(L)", 1.0);
pc.propellant.addFuel("H2(L)", 1.0);
pc.optimize = false;

cc = new Object();
cc.pc = 1;
cc.pc_unit = "MPa"
cc.ions = true;
cc.multiphase = false;
cc.condensed = false;
cc.solveNozzleFlow = true;
cc.nozzleInlet_Ar = 2.5;
cc.freezing_Ar = 2;
cc.nozzleExit_Ar = 20;

tp = new TheoreticalPerformance(pc, cc);
tp.solve();
tp.printResults();

```

```

propellant = new Propellant(2.6);
propellant.addOxidizer("O2(L)", 1.0);
propellant.addFuel("H2(L)", 1.0);

pc = {
  "propellant":propellant,
  "optimize":false
};

cc = {
  "pc":1,
  "pc_unit": "MPa",
  "ions": true,
  "multiphase": false,
  "condensed": false,
  "solveNozzleFlow": true,
  "nozzleInlet_Ar": 2.5,
  "freezing_Ar": 2,
  "nozzleExit_Ar":20
};

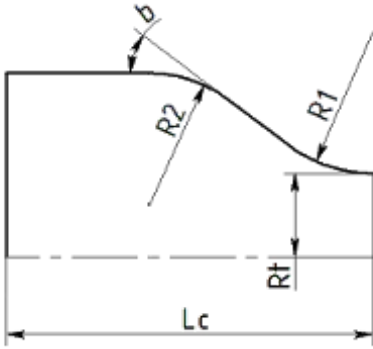
```

```
tp = new TheoreticalPerformance(pc, cc);
tp.solve();
tp.printResults();
```

Object ChamberDesignConfiguration

Object *ChamberDesignConfiguration* is a pure JavaScript object with properties and without functions. It is used to prepare the configuration parameters to be passed to constructor of objects *ChamberDesign*.

Property	Description
performance	Reference to object <i>TheoreticalPerformance</i> .
n	Number of chambers. Optional. Default value "1".
sizing_type	Type of chamber sizing. Possible values: "THRUST" "TOTAL_MASS_FLOW_RATE" "NOZZLE_THROAT_DIAMETER"
thrust	Thrust value. Used with sizing_type="THRUST".
thrust_unit	Thrust unit. One of "kN", "N", "kg", "lbf"
pa	Ambient pressure for specified thrust value.
pa_unit	Ambient pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".
mdot	Propellant total mass flow rate. Used with sizing_type="TOTAL_MASS_FLOW_RATE"
mdot_unit	Mass flow rate unit. One of "kg/s", "lbm/s".
Dt	Throat diameter. Used with sizing_type="NOZZLE_THROAT_DIAMETER"
Dt_unit	Throat diameter unit. One of "m", "mm", "in", "ft".
length_type	Type of specified chamber length. Possible values: "L_STAR" "L_C"
Lstar	Value of L*. Used with length_type="L_STAR".
Lstar_unit	Unit of L*. One of "m", "mm", "in", "ft".
Lc	Value of Lc – absolute length of combustion chamber from injector to nozzle throat. Used with length_type="L_C".
Lc_unit	Unit of Lc. One of "m", "mm", "in", "ft".
b	Contraction angle
b_unit	Contraction angle unit. One of "degrees", "radians".
R1_to_Rt	R1/Rt
R1	The same as R1_to_Rt. Note: R1 is not an absolute value.
R2_to_R2max	R2/R2max
R2	The same as R2_to_R2max. Note: R2 is an absolute value.

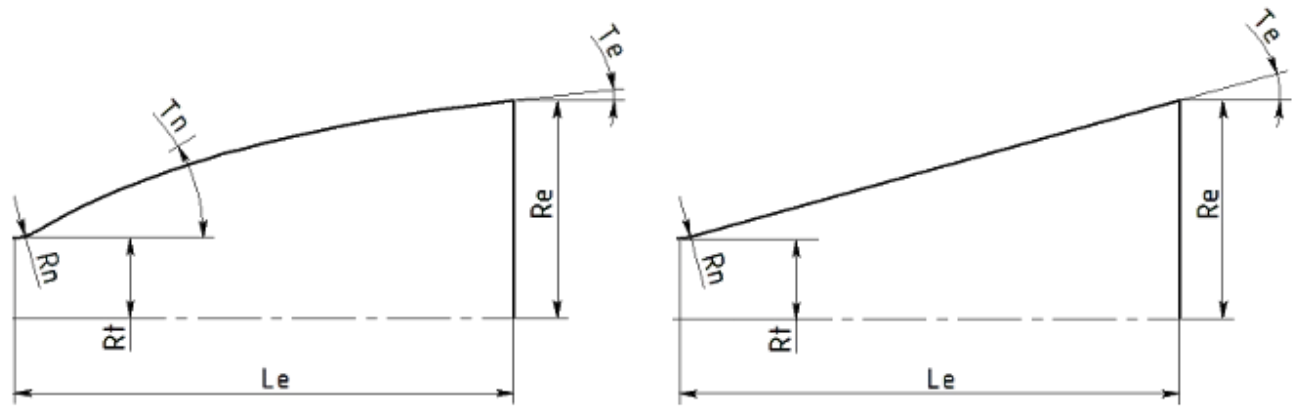


Chamber design configuration parameters

Object NozzleDesignConfiguration

Object *NozzleDesignConfiguration* is a pure JavaScript object with properties and without functions. It is used to prepare the configuration parameters to be passed to constructor of objects *NozzleDesign*.

Property	Description
chamberDesign	Reference to object <i>ChamberDesign</i> .
type	Nozzle type. Possible values: "CONICAL" "PARABOLIC" "MOC"
theta_1	Nozzle initial half-angle T_n . Used with type="PARABOLIC".
theta_1_unit	Angle unit. One of "degrees", "radians".
theta_e	Nozzle exit half-angle T_e . Used with type="CONICAL" or type="PARABOLIC".
theta_e_unit	Angle unit. One of "degrees", "radians".
R1_to_Rt	R_n/R_t
R1	The same as R1_to_Rt. Note: R1 is not an absolute value.
Le_to_Dt	Relative nozzle length Le/D_t . Optional. Used with type="MOC", mocType="FIXED_EXIT_AREA_AND_LENGTH".
Le_r	Relative nozzle length Le/Le_{con_15} (%). Optional. Used with type="PARABOLIC".
mocType	Type of MOC nozzle. Possible values: "FIXED_EXIT_AREA" "FIXED_EXIT_AREA_AND_LENGTH"
Tw_to_T0	Average T_w/T_0 . Used with type="MOC".
Tw_bar	The same as Tw_to_T0.



Parabolic and MOC nozzle design configuration parameters

Conical nozzle design configuration parameters

Object *ChamberDesignResults*

Object *ChamberDesignResults* is a pure JavaScript object with properties and without functions. It is used to return the results from solved object *ChamberDesign*.

Property	Description
Ac	Combustion chamber area.
Ac_unit	"m ² "
At	Throat area.
At_unit	"m ² "
Vc	Volume of combustion chamber.
Vc_unit	"m ³ "
Sc	Surface area of combustion chamber.
Sc_unit	"m ² "
mdot	Chamber total mass flow rate.
mdot_unit	"kg/s"
F	Chamber thrust.
F_unit	"N"
pa	Ambient pressure for calculated thrust.
pa_unit	"Pa"
Dc	Combustion chamber diameter.
Dc_unit	"m"
Lcyl	Length of cylindrical part of chamber.
Lcyl_unit	"m"
Lstar	L*

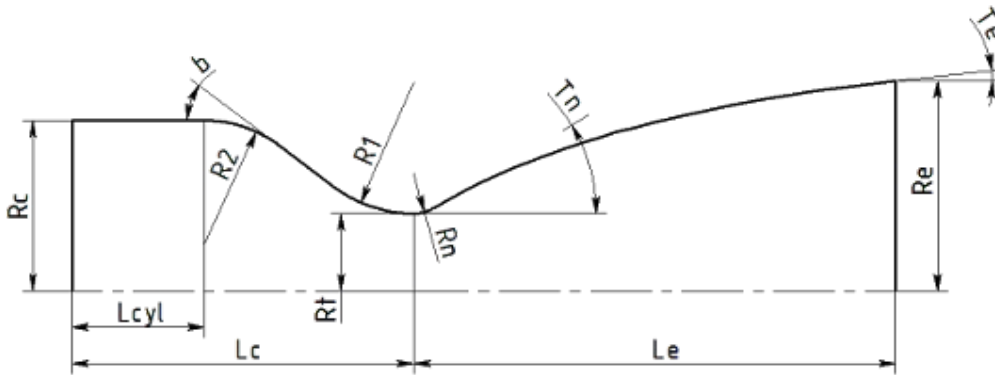
Property	Description
Lstar_unit	"m"
b	Nozzle inlet contraction angle.
b_unit	"radians"
R1	See image "Thrust chamber design result parameters".
R1_unit	"m"
R2	See image "Thrust chamber design result parameters".
R2_unit	"m"
Linlet	Length of nozzle inlet.
Linlet_unit	"m"
Vinlet	Volume of nozzle inlet.
Vinlet_unit	"m^3"
Sinlet	Surface area of nozzle inlet.
Sinlet_unit	"m^2"
Dt	Throat diameter.
Dt_unit	"m"
n	Number of thrust chambers.
contour	Array of contour points. Each element of array is a JavaScript object with properties: x - axial distance from injector plate r - radius unit - unit, "m"

Object *NozzleDesignResults*

Object *NozzleDesignResults* is a pure JavaScript object with properties and without functions. It is used to return the results from solved object *NozzleDesign*.

Property	Description
type	Type of nozzle contour. One of "CONICAL", "PARABOLIC", "MOC"
Ar	Nozzle area ratio A_e/A_t .
At	Nozzle throat area.
At_unit	"m^2"
Ae	Nozzle exit area.
Ae_unit	"m^2"
Dt	Nozzle throat diameter.
Dt_unit	"m"
De	Nozzle exit diameter.
De_unit	"m"

Property	Description
L	Nozzle length.
L_unit	"m"
theta_1	Nozzle initial half-angle T_n .
theta_1_unit	T_n _unit, "radians".
theta_e	Nozzle exit half-angle T_e .
theta_e_unit	"radians"
R1	R_n , see image "Thrust chamber design result parameters".
R1_unit	R_n _unit, "m"
contour	Array of contour points. Each element of the array is a JavaScript object with properties: - x - axial location - r - radius - unit - unit, "m" The value of axial location depends on parameter startFromLc (see function NozzleDesign.getSize()). If "false", the location of the first contour point is 0. Otherwise, the location of the first point is "Lc" (combustion chamber length).



Thrust chamber design result parameters

Object HeatTransferConfiguration

Object *HeatTransferConfiguration* is a pure JavaScript object with properties and without functions. It is used to prepare the configuration parameters to be passed to constructor of objects *HeatTransferAnalysis*.

Property	Description
chamberDesign	Reference to object <i>ChamberDesign</i> .
nozzleDesign	Reference to object <i>NozzleDesign</i> .

Rocket Propulsion Analysis v.4.0.0

Property	Description
throttle_value	Thrust chamber throttle value. Optional. Default value is "1.0".
points	Number of points along the chamber to calculate heat transfer. Optional. Default value is "50".
applyCooling	If "true", calculate wall cooling and find steady state wall temperature and heat transfer at each wall point. Otherwise, estimate heat transfer at fixed wall temperature <i>Twg</i> without applying wall cooling.
Twg	Fixed wall temperature. Used with parameter <code>applyCooling="false"</code> .
Twg_unit	Fixed temperature unit. One of "K", "F", "C",
approach	Heat transfer relations. Possible values: "IEVLEV" "BARTZ" "MIXED"
wallLayer	Relative thickness of near-wall layer. Optional. Default value "0.025".
withRadiationHeatTransfer	If "true", estimate radiation heat transfer from combustion products to gas-side wall surface.
eps_w	Emissivity of gas-side wall surface. Optional. Default value is "0.8".
applyFilmCooling	If "true", specified film slot configurations will be applied during wall cooling analysis.
designRegenCoolingJackets	If "true", the code will try to design the geometry of configured regenerative cooling segments.
filmCoolingSlots	Array of film cooling slots. Contains objects <i>FilmCoolingSlot</i> .
wallCoolingSegments	Array of wall cooling segments. Contains objects <i>WallCoolingSegment</i> .

Object *FilmCoolingSlot*

Property	Description
coolant	Object <i>Mixture</i> to specify the coolant for the film slot.
mdot_bar	Slot relative mass flow rate. The value is relative to the chamber total mass flow rate: $\text{mdot_bar} = \text{mdot_slot} / \text{mdot_total}$.
location	Slot location: distance from the injector along the longitudinal axis of the chamber.
location_unit	Location unit. One of "m", "mm", "in", "ft".

Object *WallCoolingSegment*

Rocket Propulsion Analysis v.4.0.0

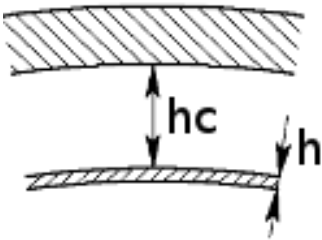
Property	Description
id	ID of the cooling slot. Any string value which is unique among all other segments.
location	Segment location: distance from the injector along the longitudinal axis of the chamber to the segment end closest to the injector.
location_unit	Location unit. One of "m", "mm", "in", "ft".
h	Thickness of the inner (gas-side) wall, or thickness of the tube wall.
h_unit	Thickness unit. One of "m", "mm", "in", "ft".
lambda	Thermal conductivity of inner (gas-side) wall.
lambda_unit	Thermal conductivity. One of "W/(m K)", "Btu/(s in R)", "Btu/(hr ft R)".
bcH	Thickness of barrier coating on the gas-side of the wall. Optional.
bcH_unit	Thickness unit. One of "m", "mm", "in", "ft".
bcLambda	Thermal conductivity of barrier coating on the gas-side of the wall.
bcLambda_unit	Thermal conductivity. One of "W/(m K)", "Btu/(s in R)", "Btu/(hr ft R)".
type	Possible values: "RADIATION" "COAXIAL" "TUBULAR" "CHANNEL"
eps_w	Emissivity of outer surface of the wall. Used with type="RADIATION". Optional, Default value is "0.8".
hc	Distance between shells. Used with type="COAXIAL".
hc_unit	Distance unit. One of "m", "mm", "in", "ft".
d1	Tube diameter (before forming) at segment end closest to the injector (see fig. "Stations of cooling segments"). Used with type="TUBULAR".
d1_unit	Diameter unit. One of "m", "mm", "in", "ft".
d2	Tube diameter (before forming) at segment end which is most distant from the injector (see fig. "Stations of cooling segments"). Used with type="TUBULAR".
d2_unit	Diameter unit. One of "m", "mm", "in", "ft".
dmin	Tube diameter (before forming) at segment end at the lowest area ratio (see fig. "Stations of cooling segments"). Optional. Used with type="TUBULAR".
dmin_unit	Diameter unit. One of "m", "mm", "in", "ft".
n	Number of tubes or channels. Used with type="TUBULAR" and type="CHANNEL".
helix	If "true", tubular segment has a helix structure. Optional. Default value is "false". Used with type="TUBULAR".
hc1	Rib height at segment end closest to the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".
hc1_unit	Height unit. One of "m", "mm", "in", "ft".
hc2	Rib height at segment end which is most distant from the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".

Rocket Propulsion Analysis v.4.0.0

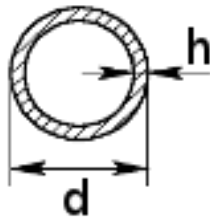
Property	Description
hc2_unit	Height unit. One of "m", "mm", "in", "ft".
hcmín	Rib height at segment end at the lowest area ratio (see fig. "Stations of cooling segments"). Optional. Used with type="CHANNEL".
hcmín_unit	Height unit. One of "m", "mm", "in", "ft".
a1	Width of channel at segment end closest to the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".
a1_unit	Width unit. One of "m", "mm", "in", "ft".
a2	Width of channel at segment end which is most distant from the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".
a2_unit	Width unit. One of "m", "mm", "in", "ft".
amin	Width of channel at segment end at the lowest area ratio (see fig. "Stations of cooling segments"). Optional. Used with type="CHANNEL".
amin_unit	Width unit. One of "m", "mm", "in", "ft". Used with type="CHANNEL".
gamma	Angle between channel passage and the plane perpendicular to symmetry axis of the chamber. Optional. Used with type="CHANNEL".
gamma_unit	Angle unit. One of "degrees", "radians".
b1	Width of rib at segment end closest to the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".
b1_unit	Width unit. One of "m", "mm", "in", "ft".
b2	Width of rib at segment end which is most distant from the injector (see fig. "Stations of cooling segments"). Used with type="CHANNEL".
b2_unit	Width unit. One of "m", "mm", "in", "ft". Used with type="CHANNEL".
bmin	Width of rib at segment end at the lowest area ratio (see fig. "Stations of cooling segments"). Optional. Used with type="CHANNEL".
bmin_unit	Width unit. One of "m", "mm", "in", "ft".
channel_type	Possible values: "CONST_CHANNEL_WIDTH" "CONST_RIB_WIDTH" If channel_type="CONST_CHANNEL_WIDTH", parameters a1, a2, and amin are used, while parameters b1, b2, and bmin are ignored. If channel_type="CONST_RIB_WIDTH", parameters b1, b2, and bmin are used, while parameters a1, a2, and amin are ignored. Used with type="CHANNEL".
lambda_rib	Optional.
lambda_rib_unit	Optional.
inputFrom	Coolant from regenerative cooling segment with specified ID.
coolant	Object <i>Mixture</i> to specify the coolant. Not applicable if inputFrom is specified.
mdot_bar	Relative mass flow rate. The value is relative to the chamber total mass flow rate: $\text{mdot_bar} = \text{mdot_slot}/\text{mdot_total}$. Not applicable if inputFrom is

Rocket Propulsion Analysis v.4.0.0

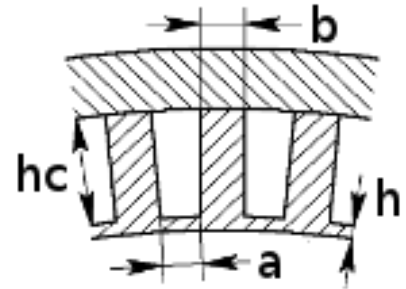
Property	Description
	specified.
coolant_p	Coolant inlet pressure. Not applicable if inputFrom is specified.
coolant_p_unit	Pressure unit. One of "Pa", "psi", "atm", "bar", "at", "MPa".
coolant_T	Coolant inlet temperature. Not applicable if inputFrom is specified.
coolant_T_unit	Temperature unit. One of "K", "F", "C",
oppositeFlow	Optional. Default value is "false".
twoPass	Optional. Default value is "false".
roughnessH	Gas-side wall average roughness height. Optional.
roughnessH_unit	Roughness height unit. One of "m", "mm", "in", "ft".
roughnessT	Gas-side wall average roughness step. Optional.
roughnessT_unit	Roughness step unit. One of "m", "mm", "in", "ft".



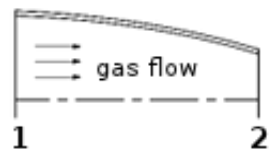
Coaxial shell segment configuration parameters



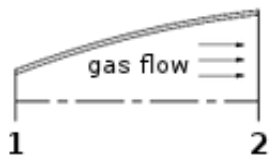
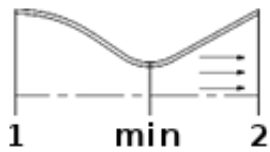
Tubular segment configuration parameters



Channel segment configuration parameters



Stations of cooling segments



Object *HeatTransferAnalysisResults*

Property	Description
approach	Heat transfer relations. Possible values: "IEVLEV" "BARTZ" "MIXED"
throttle_level	Thrust chamber throttle value.
Tw_bar	Average Tw/T0.
maxPass	MaxPass=2, if at least one regenerative cooling segments configured with two-pass coolant flow. Otherwise, maxPass=1.
units	JavaScript object with units used for values in <i>HeatTransferAnalysisResults</i> .
stations	JavaScript array with stations. Each element of the array is a JavaScript object <i>HeatTransferStation</i> with parameters for specific station of thrust chamber.

Object *Units*

Object *Units* is a pure JavaScript object with units used for values in *HeatTransferAnalysisResults*.

Property	Description
dimension	Unit for dimensions (location, diameter, length, width, etc.) Default "m".
area	Unit for area. Default "m^2".
velocity	Unit for velocity. Default is "m/s".
density	Unit for volumetric mass density. Default is "kg/m^3".
temperature	Unit for temperature, Default is "K".
pressure	Unit for pressure. Default is "Pa".
heat_flux	Unit for heat flux. Default is "W/m^2".
convective_heat_coefficient	Unit for convective heat transfer coefficient. Default is "W/m^2-K".
alpha	The same as "convective_heat_coefficient"

Object *HeatTransferStation*

Object *HeatTransferStation* is a pure JavaScript object with result parameters for specific station of thrust chamber.

Property	Description
x	Axial location.
r	Radius of thrust chamber contour.
q_total	Total heat transfer.
q_rad	Radiation heat transfer.

Property	Description
q_conv	Convective heat transfer.
alpha	Convective heat transfer coefficient.
Twg	Gas-side wall temperature.
cooling	JavaScript array. Each element of the array is a JavaScript object <i>WallCooling</i> representing parameters of wall cooling. For regenerative cooling segments configured with two-pass coolant flow, the array contains two elements. For regenerative cooling segments configured with one-pass coolant flow, as well as for segments with radiation cooling, the array contains one element.

Object *WallCooling*

Object *WallCooling* is a pure JavaScript object with parameters for specific wall cooling segment.

Property	Description
id	Segment id.
type	Segment type. Possible values: "radiation" "radiation, film slot" "regen coaxial" "regen coaxial, film slot" "regen tubular" "regen tubular 1" "regen tubular 2" "regen tubular, film slot" "regen tubular 1, film slot" "regen tubular 2, film slot" "regen channel" "regen channel 1" "regen channel 1" "regen channel, film slot" "regen channel 1, film slot" "regen channel 1, film slot" "failed"
Tw _i	Temperature on the wall under the thermal barrier coating layer.
Tw _c	Temperature of the wall on the coolant side.
T _c	Coolant temperature.
p _c	Coolant pressure.
w _c	Coolant velocity.
ρ _h _c	Coolant volumetric mass density.
A	Coolant passage flow area.
d _e	Equivalent diameter of the coolant passage.
a, h _c	Dimensions of the coolant passage. For coaxial shell: n.a.

Property	Description
	For tubular cooling jacket: dimensions of deformed tube. For channel cooling jacket: dimensions of channel.
n	Number of tubes or channels.
helix	Angle of tube or channel having helix structure.
h	Thickness of the inner (gas-side) wall, or thickness of the tube wall.
bch	Thickness of barrier coating on the gas-side of the wall. 0 if not applicable.

Attachments

Equivalence Ratio calculation

$$v_{ox} = \sum_{i=1}^N \left[\begin{array}{l} \text{if } (v_i < 0) \text{ then } (n_i v_i) \\ \text{else } 0 \end{array} \right]$$

$$v_{rd} = \sum_{i=1}^N \left[\begin{array}{l} \text{if } (v_i > 0) \text{ then } (n_i v_i) \\ \text{else } 0 \end{array} \right]$$

$$ER = -\frac{v_{ox}}{v_{rd}}$$

where

- N - number of chemical elements in chemical formula
- i - index of chemical element in chemical formula (1, 2, ...)
- n_i - number of atoms of chemical element i in chemical formula
- v_i - valency of chemical element i
- v_{ox} - sum of oxidizing valencies in chemical formula
- v_{rd} - sum of reducing valencies in chemical formula

Examples:

H_2O (water)

$N = 2$ (H , O)

for $i = 1$ (H):

$n_1 = 2$

$v_1 = 1$ (reducing)

for $i = 2$ (O):

$n_2 = 1$

H_2O_2 (hydrogen peroxide)

$N = 2$ (H , O)

for $i = 1$ (H):

$n_1 = 2$

$v_1 = 1$ (reducing)

for $i = 2$ (O):

$n_2 = 2$

$$v_2 = -2 \text{ (oxidizing)}$$

$$v_{ox} = n_2 v_2 = 1 \times (-2) = -2$$

$$v_{rd} = n_1 v_1 = 2 \times 1 = 2$$

$$ER = -\frac{-2}{2} = 1$$

$$v_2 = -2 \text{ (oxidizing)}$$

$$v_{ox} = n_2 v_2 = 2 \times (-2) = -4$$

$$v_{rd} = n_1 v_1 = 2 \times 1 = 2$$

$$ER = -\frac{-4}{2} = 2$$

Oxygen Balance calculation

$$b_{ox} = \frac{M_{ox}}{v_{ox} M} \sum_{i=1}^N n_i v_i$$

where

$$M_{ox} = 15.99 \quad \text{- atomic weight of oxygen}$$

$$v_{ox} = -2 \quad \text{- valency of oxygen}$$

$$N \quad \text{- number of chemical elements in chemical formula}$$

$$i \quad \text{- index of chemical element in chemical formula (1, 2, ...)}$$

$$n_i \quad \text{- number of atoms of chemical element } i \text{ in chemical formula}$$

$$v_i \quad \text{- valency of chemical element } i$$

$$M \quad \text{- molecular weight of chemical specie}$$

Examples:

$$NaClO_4 \text{ (Sodium Perchlorate)}$$

$$N = 3 \text{ (} Na, Cl, O \text{)}$$

for $i = 1 \text{ (} Na \text{)}$:

$$n_1 = 1$$

$$v_1 = 1$$

$$M_1 = 22.99$$

for $i = 2 \text{ (} Cl \text{)}$:

$$n_2 = 1$$

$$v_2 = -1$$

$$M_2 = 35.45$$

for $i = 3 \text{ (} O \text{)}$:

$$n_3 = 4$$

$$v_3 = -2$$

$$M_3 = 15.99$$

$$M = n_1 M_1 + n_2 M_2 + n_3 M_3 =$$

$$22.99 + 35.45 + 4 \times 15.99 = 122.4$$

$$Al \text{ (Aluminum)}$$

$$N = 1 \text{ (} Al \text{)}$$

for $i = 1 \text{ (} Al \text{)}$:

$$n_1 = 1$$

$$v_1 = 3$$

$$M_1 = 26.98$$

$$M = n_1 M_1 = 26.98$$

$$b_o = \frac{15.99}{-2 \times 26.98} (1 \times 3) =$$

$$-0.29633 \times 3 = -0.88899$$

$$b_o = \frac{15.99}{-2 \times 122.4} (1 \times 1 + 1 \times (-1) + 4 \times (-2)) = -0.13063 (1 - 1 - 8) = 0.52255$$